Empirical Software Engineering (2026) 31:116
https://doi.org/10.1007/s10664-026-10853-z

Do code LLMs do static analysis?

Chia-Yi Su[1] · **Collin McMillan**[1]

Received: 17 May 2025 / Accepted: 26 March 2026 / Published online: 10 April 2026

Abstract
This paper investigates code LLMs' capability of static analysis during code intelligence tasks such as code summarization and generation. Code LLMs are now household names for their abilities to do some programming tasks that have heretofore required people. The process that people follow to do programming tasks has long been understood to require static analysis. For example, human programmers navigate the call graph of large programs to comprehend the different parts of those programs. Education in programming includes static analysis under the assumption that better static analysis skills beget better programming. While popular culture is replete with anthropomorphic references such as LLM "reasoning", in fact code LLMs could exhibit a wholly alien thought process to humans. This paper studies the specific question of static analysis by code LLMs. We use three different static analysis tasks (callgraph generation, AST generation, and dataflow generation) and three different code intelligence tasks (code generation, summarization, and translation) with two different open-source models (Gemini and GPT-4o) and closed-source models (CodeLlaMA and Jam) as our experiments. We found that LLMs show poor performance on static analysis tasks and that pretraining on the static analysis tasks does not generalize to better performance on the code intelligence tasks and vice versa.

Keywords Static analysis · Code intelligence tasks · Large language models · LLM reasoning

Communicated by: Yu Huang, Antonino Ferraro

This article belongs to the Topical Collection: *SI: Human-Centered AI for SE*.

✉ Chia-Yi Su
csu3@nd.edu

[1] Department of Computer Science and Engineering, University of Notre Dame, Notre Dame, IN, USA

1 Introduction

Popular discussion around large language models (LLMs) is replete with anthropomorphic references such as LLM "reasoning" and "creativity" (Tian et al. 2024; Zhao et al. 2024). Even dialogue in scientific literature uses biological terms such as "attention" and "hallucination" to refer to parts of LLM architecture or behavior. The implication is that LLMs think and act like people, with human intentions and human thought processes. One may conclude that this implication is clearly hyperbole – many authors argue that impressive LLM abilities are in fact merely pattern matching across big data input (Wang et al. 2023) and/or carefully curated finetuning (Chen and Mueller 2024). Yet the processes that LLMs follow are controversial and there is not yet sufficient empirical evidence on specific LLM behaviors to draw strong conclusions.

One group of LLM behaviors in the spotlight is in software development tasks. Code LLMs are language models specializing in these tasks such as generating code samples from natural language prompts, code comment generation, and translation from one programming language to another. Many programmers use code LLMs every day as a normal part of their workflow. For example, Google estimated in November of 2024 that over 25% of its new code is written by AI and later reviewed by engineers (arstechnica 2024). The rapid rise of LLM-generated code has led to anxiety about the safety and security of programs containing that code (Mohsin et al. 2024; Wang et al. 2024b), privacy and other legal issues (He et al. 2024), and even the future of software engineering as a career (Rasnayaka et al. 2024; Tona et al. 2024). Knowing how code LLMs do coding tasks could shed light on these concerns.

Meanwhile, a fundamental part of what people do during programming tasks is static analysis. Static analysis is the process of examining code without executing it (Ayewah et al. 2008). Studies of human programmers have for decades shown how people build a mental model of program behavior through static tracing of three relationships in particular: function invocations (i.e., function call relationships), data flow among variables, and the hierarchical structure of code elements, such as how expressions, statements, and blocks are organized (Littman et al. 1987). A rich body of literature describes these and similar static analysis tasks which underpin development of static analysis tools built into popular IDEs (Luo et al. 2021).

This paper asks the question: do code LLMs do static analysis? Humans do static analysis as a core part of software development, so if code LLMs follow a process similar to humans, then one would expect code LLMs to do static analysis. We conduct experiments in which we 1) observe the performance of LLMs on software development tasks, 2) observe the performance of those same LLMs on static analysis tasks, 3) finetune open-source LLMs to do static analysis tasks, 4) observe if the finetuned LLMs' performance on development tasks changes, and 5) observe if LLMs have the byproduct as the results of being trained on code intelligence tasks. One would expect improved performance on development tasks if the code LLMs are trained to do static analysis better and have static analysis as a byproduct.

We found that code LLMs generally do a poor job at static analysis tasks but have strong performance at coding tasks. We also found that finetuning LLMs with static analysis tasks improved models' ability to do simple static analysis tasks, but did not improve models' ability on more complicated static analysis and coding tasks. In addition, training LLMs on code intelligence tasks does not lead to better generation of static analysis tasks. We found these results over two open-source (CodeLlaMA, jam) and two closed-source mod-

els (GPT-4o, Gemini) and two programming languages (Java, C/C++). We studied three static analysis tasks (call graph, data flow graph, and abstract syntax tree generation) and three programming tasks (code summarization, code translation, and code generation). Our findings contribute to the ongoing academic debate about how LLMs function and have implications for task generalization, training procedures, and the design of future LLMs for software engineering tasks.

2 Background and Related Work

This section discusses studies of program comprehension and the role static analysis plays in that comprehension process, code large language models, and studies of capabilities of those models.

2.1 Static Analysis in Program Comprehension

This section discusses the studies of program comprehension and static analysis for program comprehension.

Empirical Studies Program comprehension has been a target of empirical studies for a decade. Several empirical studies have been conducted to understand how programmers comprehend code. Littman et al. (1987) found that programmers use data flow for comprehending code. Pennington (1987) also found that programmers combine data flow and control flow to comprehend source code in their study with 80 professional programmers. Shneiderman and Mayer (1979) have similar conclusion that syntactic information is as important as semantic information. The way that programmers represent the code mentally can be referred as mental models (Siegmund 2016; O'brien 2003; Storey 2005; Harth and Dugerdil 2017). Harth and Dugerdil (2017) classified this into three different periods: 1) classical period 2) optimistic period 3) pragmatic period. In the classical period, several different strategies on how programmers understand code were proposed. During the optimistic period, several tools and studies were developed and conducted. The pragmatic period focus more on measurement instead of creating new methods.

Static Analysis Various studies have established that three static analysis tasks in particular are crucial to program comprehension: 1) call graph analysis, 2) data flow analysis, and 3) Abstract Syntax Tree (AST) generation. For example, Jiang et al. (2017) conducted extensive studies on impact analysis tasks. They recruited nine professional programmers for a depth study and 35 programmers for the breath study. They found that callgraphs help programmers to navigate the source code. Wallace et al. (2025) conducted a project-level eye-tracking study with 10 Java programmers for source code summarization. They suggested using callgraph to develop context-aware LLMs for source code summarization. Konopka et al. (2018) conducted an empirical study with novice and intermediate programmers and showed the importance of using data flow analysis for program comprehension tasks. In addition, they proposed a data-flow based metrics to evaluate how human programmers comprehend a program. Bergeretti and Carré (1985) also show that the information flow helps various stages of software development. Maalej et al. (2014) conducted a study on how software developers comprehend source code with 28 software developers. They found that some participants drew the data flow to comprehend the program. Hov-

emeyer et al. (2016) showed that simple AST helps to understand the strategies that students use for their programming assignments. Agrahari and Chimalakonda (2020) show that AST helps the novice programmers to learn the fundamental of data structure. Schanzer et al. (2019) developed an AST-based tool for vision impaired programmers. They found the improvement with their tools. Overall, these show that programmers rely on static analysis to do program comprehension tasks.

Static analysis helps programmers analyze code base without the need to execute the source code and any input. Different theories on static analysis have been proposed. Park et al. (2021) proposed the parametric static analysis to theorize the static analysis. Static analysis helps programmers in different aspects of software development (Vassallo et al. 2020; Beller et al. 2016). Vassallo et al. (2018) found that developers use the warning message from automatic static analysis tools to find the bugs in the source code. Ayewah et al. (2008) combined the static analysis tools for bug detection. Static analysis can be used to aid the program comprehension (Eisenbarth et al. 2001). In the study conducted by Tymchuk et al. (2018), they showed that static analysis tools can also be served as an educational tool. Singh et al. (2017) showed that static analysis tools can reduce the workload of code review.

2.2 Code Large Language Models

Code LLMs have been the trend of in the recent year (Zhang et al. 2023; Jiang et al. 2024a; Chen et al. 2025) and have been applied to several different software engineering tasks (Nam et al. 2024; Gu 2023; Zheng et al. 2025; He et al. 2025; Zhong and Wang 2024). In this section, we discuss the history of code LLMs. Specifically, we focus on how LLMs are trained and how the source code is represented during different stages. Then, we discuss the application of LLMs in software engineering and studies on the capabilities of code LLMs.

History Sun et al. (2024a) divided the history of code LLMs into three groups: 1) neural language models for code 2) code pre-trained models 3) code LLMs. Natural language models represent the code as the natural language. Different methods have been proposed to represent code. Alon et al. (2019b) encode each code token as a word vector. Zhang et al. (2019) showed that ASTs can better encode the information in the code. LeClair et al. (2020) applied graph neural network to represent code for code summarization. Code pre-trained models pretrain the language models with huge amount of source code and finetuned it with downstream tasks. Wang et al. (2021) pretrained the models with encoder-decoder transformer architectures. Li et al. (2023b) proposed a decoder architecture only model for pretrainig. More recently, the attention shift toward scaling up the models and in-context learning. The models include open source models Codellama (Roziere et al. 2023) and several closed source models (Brown et al. 2020; Anil et al. 2023).

Applications LLMs have been applied to different domains of software engineering tasks (Zheng et al. 2025, 2023; Jin et al. 2024). Sarsa et al. (2022) and Sun et al. (2023b) examine the capabilities of LLMs on code summarization. Khan and Uddin (2022) applied LLMs to generate the summarization. Koziolek et al. (2024) leveraged the LLMs and retrieval augmentation for code generations. Different evaluation methods have also been studied (Liu et al. 2024c, b). Yuan et al. (2024) proposed multi-agent systems for code translation. Pan et al. (2023) applied decoder-only LLMs to translate different programming languages into Python. In addition to program comprehension tasks, Xue et al. (2024) used

LLMs to generate the test case for FinTech software testing. Lu et al. (2024a) use LLMs for vulnerabilities detection with in-context learning. Yin et al. (2024) and a LLM-based framework for automatic program repair. Overall, LLMs are widely used in the software engineering.

Capability Studies Several studies on LLMs capability have been published (Yang et al. 2024; Li et al. 2024b; Jiang et al. 2024a; Fan et al. 2024). Yang et al. (2024) divided the capabilities of code LLMs into three different ares 1) programming skills 2) complex reasoning 3) structure knowledge. Li et al. (2022) showed that LLMs defeat around half of the programmers during programming competitions. Several studies have showed the promising results on using multiple LLMs as an agent for code generation (Liu et al. 2023a, b; Talebirad and Nadiri 2023). Li et al. (2023a) showed that LLMs can do complex reasoning on debugging. Chen et al. (2022) showed that LLMs can reason the output of the code for several math problems. Although several papers showed promising results on reasoning, several studies have also showed that LLMs can only do reasoning when the code is simple (Liu et al. 2024a; Chen et al. 2024). Madaan et al. (2022) and Wang et al. (2022) showed the ability of LLMs for structure prediction after few-shot learning.

3 Experimental Design

Our experiments center around four topics: 1) we observe LLM performance on code development tasks, 2) we observe LLM performance on static analysis tasks in baseline, in-context learning setting, and finetuned setting, 3) we finetune LLMs to do static analysis and determine if those models' performance on code development tasks increases, and 4) we observe whether LLMs can do static analysis as a byproduct of being trained on code development tasks. This section covers our research questions, methods to answer those questions, and experimental settings.

We divide our experiments across open-source and closed-source LLMs. For open-source LLMs we can closely control experimental variables and conduct finetuning. For closed-source LLMs we can access the largest available models, but we give up control over many variables and the ability to finetune.

3.1 Research Questions

Our research objective is to determine if LLMs that do code development tasks also "know how" to do static analysis and to determine if models' ability to do static analysis is connected to the models' ability to do code development tasks and vice versa. We ask the following Research Questions (RQs):

RQ1 What is the performance of closed-source LLMs on static analysis tasks as is?

RQ2 What is the performance of open-source LLMs on static analysis tasks when finetuned for those tasks?

RQ3 What is the performance of open-source LLMs on code development tasks when finetuned for those tasks, after they have been finetuned for static analysis?

RQ4 What is the performance of open-source LLMs on static analysis tasks after they have been finetuned for code intelligence tasks?

RQ5 What are the common error types in static analysis?

The rationale behind RQ1 is that state-of-the-art commercial LLMs are already known to perform code development tasks well (Zheng et al. 2025), but we do not yet know how well these models perform static analysis tasks. The rationale behind RQ2 is that LLMs may be able to perform static analysis tasks if they are explicitly trained to do so, and the availability of open-source models provides us the opportunity to do this training. The rationale behind RQ3 is to examine whether finetuning the models with static analysis tasks could improve the performance of code intelligence tasks. This is because human programmers use various static analysis tasks as an intermediate thought process for code intelligence tasks (Refer to Section 2.1). If a model truly has the anthropomorphic characteristic, static analysis should provide the fundamental knowledge for LLMs to perform code intelligence tasks. The rationale for RQ4 is that models can internalize some aspects of static analysis after finetuning models on code intelligence tasks as a byproduct if models can do static analysis tasks. This research question allows us to have a more thorough evaluation on whether LLMs can do static analysis tasks. The rationale for RQ5 is that LLMs may fall short on specific areas. RQ5 provides the insights on where the LLMs fall short on static analysis tasks.

3.2 Code Development Tasks

In this section, we discuss the three different code development tasks that we use i.e. code summarization, code translation, and code generation and the datasets for each task. We follow the instructions from the dataset because our idea is to show whether LLMs follow what human programmers do for code development tasks. We use C/C++ and Java as an example.

Code Summarization Code summarization is the task of writing natural language descriptions of a section of code (Haiduc et al. 2010). Code summarization is important because it is the core of automatic documentation generation and related tool support for helping programmers understand code; the history of research efforts in code summarization has been chronicled in recent surveys (Zhang et al. 2024, 2022). We study this task in this paper because code summarization has become an advertised feature of code LLMs (Sun et al. 2024b) and because different studies show how people do static analysis in order to do code summarization (Rodeghero and Mcmillan 2015; Stapleton et al. 2020). In this paper, for Java we use the "170k" dataset by Su and McMillan (2024). That dataset has its origins in a study of code summarization dataset recommendations by LeClair and McMillan (2019) and further vetted by Bansal et al. (2021). A major advantage to the dataset is that it's test set corresponds to a set held out of training in the `jam` model, which strongly avoids data contamination concerns (see Section 3.4). For C/C++ we use a dataset by Liu et al. (2021) which followed similar best practices (e.g., LeClair and McMillan 2019; Allamanis 2019).

Code Translation Code translation is the task of writing a program in one language that has the same functionality as a program in another programming language. Code translation can help program comprehension by in converting pseudo code into machine readable code (Holt and Turanski 1960) and translate less-readable programming language into a more human-readable programming language (Hong and Ryu 2024). Several studies have shown that code translation requires static analysis when done manually by people (Siddiqui et al. 2023; Feautrier 1991). In this paper, we use code translation dataset in CodeXGLUE (Lu et al. 2021) as our dataset for Java. CodeXGLUE collects several datasets for code intelligence tasks. CodeXGLUE divides the datasets into code-code, code-text, text-code, and text-code. The code translation dataset that we use is collected from several public

repositories with deduplication and clear train/test/validation separation. For C/C++, we use the C to Rust dataset in Huggingface website (Yu 2023). For execution-based evaluation, we finetuned our models with CodeTransOcean (Yan et al. 2023) and test our models with datasets proposed by Pan et al. (2024) because we do not find test cases in CodeTransOcean at the time we download the dataset.

Code Generation Code generation is the task of generating source code that implements the functionality described in natural language. Code generation can be used as an educational tool to help novice programmers learn programming (Becker et al. 2023; Mosterman 2006) and is widely used to improve the productivity of software developers (Ross et al. 2023; Xu et al. 2022). Static analysis is part of generating code in that an understanding of the static structure leads to higher code quality and few bugs (Barke et al. 2023; Nachtigall et al. 2019). In this paper, we used CONCODE dataset (Iyer et al. 2018) from CodeXGLUE (Lu et al. 2021) for Java. The dataset is collected from 33,000 repositories and they filtered out the methods without Javadoc as natural language description. Although the dataset has very clear train/validation/test split, we use separate half of the validation set as our testset because we do not find the ground-truth for testset at the time we download. For C/C++, we use Xlcost (Zhu et al. 2022). Xlcost is a dataset that contains seven different programming languages and natural language, though we only use the C/C++ component. For execution-based evaluation, we finetuned our models with the programming competition dataset proposed by Li et al. (2022) and evaluated the results with pass@k and compilation rate. Note that the dataset includes the problems with several programming languages. We only used problems that have C/C++ solutions. We explain the metrics that we used in Section 3.5.

3.3 Static Analysis Tasks

In this section, we discuss three static analysis tasks in our experiments, i.e. AST, dataflow graph, and callgraph generation with two different programming languages, i.e. C/C++ and Java.

Abstract Syntax Tree Generation An Abstract Syntax Tree (AST) is a tree structure that captures the syntactic information of the source code. In addition to being a key intermediate representation in code compilation, ASTs are used to localize software bugs (Neamtiu et al. 2005), comprehend source code (Sun et al. 2023a; Welty 1997), and in software evolution (Neamtiu et al. 2005). Different studies have also shown that ASTs help program comprehension, such as when learning how to code (Agrahari and Chimalakonda 2020; Egan and McDonald 2014). In this paper, for Java we use dataset recommendation by LeClair and McMillan (2019). We use 52M to train `jam` (Su et al. 2023) and further extract 25k to train CodeLlaMA (Roziere et al. 2023) for practical reasons (refer to Section 3.4). We use the same testset (i.e., the 8k testset) proposed by LeClair and McMillan (2019). For C/C++, we extract 9k methods for test and 10k methods for training from a dataset proposed by Haque et al. (2021). We used this dataset because they followed LeClair and McMillan (2019) for preprocessing. For both C/C++ and Java, we used the srcML tool (srcml 2025) to generate AST in XML format in keeping with recommendations by Bansal et al. (2023).

Callgraph Generation Callgraph is a graph that represents the relationship between each method (Ryder 1979). Callgraphs are an important information for code comprehension source code (Alanazi et al. 2021; Walunj et al. 2019; Bansal et al. 2024; Alanazi 2021).

In this paper, we used Doxygen (doxygen 2023) to generate callgraphs for both C/C++ and Java. For C/C++ dataset, we extracted 25k methods as our testset and 20k methods as our training set from Haque et al. (2021). We generated the callgraph within a file for C/C++ to reduce the tokens of the prompt. For Java, we used the testset proposed by LeClair and McMillan (2019) and further extracted 20k methods from the rest of dataset for finetuning. We generated the callgraph from the entire project because Java does not usually have the callgraph within the file. To reduce the tokens of the prompt, we only provided caller methods in the prompt instead of the complete files.

Dataflow Graph Generation Dataflow graph represents how one variable passes to another method or variable. Dataflow graph is important because it helps programmers to do code comprehension (Sihler 2024; Kargén and Shahmehri 2012) and increase productivity of software developers (Al-Saiyd 2017). In this paper, we used Joern (joern 2025) to generate the data flow of the method. We used Joern because Joern is highly mature for Java and C/C++. For Java dataset, we used the testset recommended by LeClair and McMillan (2019) to avoid data contamination. We further extracted 15k methods other than testset as our training set. For C/C++, we extracted 8k methods as our testset and 8k methods as our training set from Haque et al. (2021). We used the same procedure to generate dataflow for both datasets.

3.4 Code Models

We use two open-source models and two closed-source models in our experiments. Our goal was to use a range of models from a small one where we can control all experimental variables, to large models that represent the state-of-the-art but where we lose control of several variables.

Open Source Models

JAM We used Jam (Su et al. 2023) with 1,024 context length throughout out entire experiments. `jam` is a language model based on GPT2 architecture for various code intelligence tasks on Java. `jam` was pretrained with 52m Java methods with clear train/validation/test set, public available dataset, and deduplication toolkit to avoid data contamination. While `jam` only has 350m parameters, Su and McMillan (2024) showed in their study that it is small enough to be useful for code intelligence tasks. The advantage of this model is that this model can be run on a single consumer hardware, e.g. 16G VRAM GPU and can be used for the experiments with more controls before scaling up to larger language models.

CodeLlaMA Instruct CodeLlaMA instruct (Roziere et al. 2023) is an open-souce model released by Meta. We used CodeLlaMA instruct because it can better understand the prompt. We used the 13B model in our experiments because it can better fit into our 24G GPU (Weyssow et al. 2025) when using Qlora (Dettmers et al. 2024). We used the Qlora released in Poludin (2025). CodeLlaMA is the trade-off between closed-sourced models and Jam because we have access to the models although we still do not have the control on the dataset used for pretraining.

Closed Source Models

GPT-4o mini GPT-4o mini is the commercial models released by OpenAI (Achiam et al. 2023). GPT-4o mini is a closed-source model namely we do not have access to the pretraining data and we send our private source code to the model through API. Because of its closed-source, we cannot avoid data contamination issues (Balloccu et al. 2024; Jiang et al.

2024b) and data privacy issues (Wu et al. 2024). Though several studies have shown the success of GPT models (Jain and Purandare 2025; Wang et al. 2024c).

Gemini Gemini is another commercial model released by Google (Team et al. 2023, 2024) and is enhanced for multimodal capability. Gemini is also a closed-source model. We used Gemini as another closed-souce model because Gemini has been compared with OpenAI's ChatGPT in several studies in both software engineering (Sobo et al. 2025; Qi et al. 2023; Siam et al. 2024; Su et al. 2024) and other research topics (Strzalkowski et al. 2024) and has shown superior than GPT in some studies (Lee et al. 2023).

3.5 Evaluation Metrics

In this section, we explain the evaluation metrics for three static analysis tasks and three code intelligence tasks.

Levenshtein distance Levenshtein distance calculates the similarity between two strings by computing the number of edits between two strings. In this paper, we used rapidfuzz implemented by Bachmann (2020) and converted the value to zero to one scale, where zero means two strings are dissimilar and one means two strings are the same.

Pair Accuracy Pair accuracy calculates the percentage of accurate edges in a predicted callgraph. For example, in a predicted callgraph with edges $E = (A, B), (B, C)$ and the correct edge $E = (A, B)$, the pair accuracy would be 0.5 because there are two edges in the predicted callgraph and only one of the predicted edges is correct.

Chain Accuracy Chain accuracy calculates the percentage of accurate callgraphs in all predicted callgraphs. For example, in a predicted callgraph with edges $E = (A, B), (B, C)$ and the true callgraph with edges $E = (A, B)$, the chain accuracy would be 0 because there are two edges in the predicted callgraph, but there is only one edge in the true callgraph.

METEOR METEOR (Banerjee and Lavie 2005) is a metric to evaluate summarization tasks that considers the word similarity rather than exact word match only. The scale of METEOR is between 0 and 1, where zero means no similarity between two sentences and one means two sentences are the same.

USE USE (Haque et al. 2022) encodes the reference summary and the predicted summary to USE score and computes the cosine similarity between reference summary and the predicted summary. Haque et al. (2022) shows that USE align better with human perspective. The scale of USE score is between -1 and 1, where -1 means there is no similarity between two summaries and 1 means two summaries are most similar.

CodeBERTScore CodeBERTScore (Zhou et al. 2023) is based on the BERTScore. CodeBERTScore encodes both natural language description and source code into a fixed length vector and computes the cosine similarity between encoded representation of the reference and the predicted tokens. In this paper, we used the package provided by Zhou et al. (2023) to compute the CoderBERTScore F1.

Pass@k pass@k (Chen 2021) evaluates how many generated samples passes test cases, where k is the number of generated samples. In this paper, we used $k = 1$ as our evaluation.

Compilation rate Unlike pass@k, compilation rate calculates how many generated samples are successfully compiled without the need to pass the test cases. The scale of the compilation rate is between 0 and 1, where zero means no generated code can be compiled and one means all of the generated code are compiled successfully.

3.6 Prompt Templates

In this section, we explain the prompt template for static analysis tasks. We used the same prompts for both Gemini and GPT models. Please note that the main purpose of this paper is to show that current LLMs are lack of reasoning skills instead of finding the prompts with the highest performance.

Template for dataflow analysis without examples We used the following prompt for dataflow analysis without examples, where {source_code} is the function for dataflow analysis. {main_ method_name} is the source of the dataflow and {sink} is the sink of the dataflow.

```
Given the code {source_code}, please do the dataflow analysis from source {
main_method_name} to sink {sink} and print the statements, but without any
explanation and markdown. Also, treat each statement as a statment instead
of a block. Return the result with the template result:<statement1> -> <
statement2> result:<statement1> -> <statement2>. Remember to use template
as well.
```

Template for dataflow analysis with example We used the following prompt for dataflow analysis with the example, where {source_code} is the function for dataflow analysis. {main_ method_name} is the source of the dataflow and {sink} is the sink of the dataflow. {example_code} is the example function and {example_data_flow} is the dataflow of the example code.

```
example_code = '''
int main(int argc, char *argv[]) {
    if (argc > 1 && strcmp(argv[1], "42") == 0) {
        fprintf(stderr, "It C!\n");
        exit(42);
    }
    printf("What is this programming language?\n");
    exit(0);
}
example_data_flow = '''
result: int main(int argc, char *argv[]) { ->
if (argc > 1 && strcmp(argv[1], "42") == 0) {
'''

Given the example code {example_code} and its dataflow analysis result {
example_data_flow} from source main to sink strcmp. Given the code {
source_code}, please do the dataflow analysis from source {main_method_name
} to sink {sink} and print the statements, but without any explanation and
markdown. Please follow the example to generate the dataflow analysis
result.
```

Template for AST generation without example We used the following prompt for AST generation without examples, where {source_code} is the function for AST generation.

```
Given the code {source_code}, please generate the complete srcml of the
given code without any explanation and markdown. Please remember to follow
srcml rules.
```

Template for AST generation with example We used the following prompt for AST generation with examples, where {source_code} is the function for AST generation. {example_code} is the example function and {example} is the srcml of the example code.

```
example = '''
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit xmlns="http://
www.srcML.org/srcML/src" revision="1.0.0" language="C" filename="test.c"><
function><type><name>int</name></type> <name>main</name><parameter_list>()
</parameter_list> <block>{<block_content><expr_stmt><expr><call><name>
printf</name><argument_list> (<argument><expr><literal type="string">"Hello
 World!"</literal> </expr></argument>)</argument_list></call></expr>;</
expr_stmt> <return>return <expr><literal type="number">0</literal></expr
>;</return></block_content>}</block></function></unit>
'''
example_code = '''
int main() {
    printf("Hello World!");
    return 0;
}
'''

Here's the example code {example_code} and the srcml of that code {example
}. Given the code {code}, please generate the complete srcml of that code.
Please remember to follow the example. Here's the srcml:
```

Template for callgraph generation without example We used the following prompt for callgraph generation without examples, where {source_co-de} is the function for callgraph generation.

```
Given the example code {example_code} and example callgraph {
example_callgraph} for the function calculate. Given the code {source_code
}, please follow the example to generate all paths of the callgraph for
method {function_name} in the format path: a->b->c etc, where -> means call
. please just generate the graph without any explanation, markdown,
additional information and remember the format is path: a->b->c
```

Template for callgraph generation with example We used the following prompt for callgraph generation with examples, where {source_code} is the function for callgraph generation. {example_code} is the source code example for generating callgraph and the {example_callgraph} is the callgraph for the {example_code}.

```
example_code = '''

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}

int calculate(int x, int y) {
    int sum = add(x, y);
    int product = multiply(x, y);
    return sum + product;
}

int main() {
    int result = calculate(3, 4);
    return result;
}
example_callgraph = '''
    path: calculate -> add
    path: calculate -> multiply
'''
Given the example code {example_code} and example callgraph {
example_callgraph} for the function calculate. Given the code {source_code
}, please follow the example to generate all paths of the callgraph for
method {function_name} in the format path: a->b->c etc, where -> means call
. please just generate the graph without any explanation, markdown,
additional information and remember the format is path: a->b->c
```

Template for AST generation after finetuning with code summarization We used the following prompt for AST generation after finetuning with code summarization, where {code} is the function for AST generation. {example_code} is the source code example for generating AST and the {example} is the AST for the {example_code}.

```
example = '''
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit xmlns="http://
www.srcML.org/srcML/src" revision="1.0.0" language="C" filename="test.c"><
function><type><name>int</name> </type> <name>main</name><parameter_list>()
</parameter_list><block>{<block_content><expr_stmt><expr><call><name>printf
</name><argument_list>(<argument><expr><literal type="string">"Hello World
!"</literal></expr></argument>)</argument_list></call></expr>;</expr_stmt><
return>return <expr><literal type="number">0</literal></expr>;</return</
block_content>}</block></function></unit>
'''
example_code = '''
int main() {
    printf("Hello World!");
    return 0;
}
'''
[INST] You have know how to describe the code. Now, here's the example code
 {example_code} and the srcml of that code {example}. can you follow the
example to generate the srcml of the {code} without explanation -- just
srcml [/INST]
```

Template for AST generation after finetuning with code generation We used the following prompt for AST generation after finetuning with code generation, where {code} is the function for AST generation. {example_code} is the source code example for generating AST and the {examp- le} is the AST for the {example_code}.

```
example = '''
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit xmlns="http://
www.srcML.org/srcML/src" revision="1.0.0" language="C" filename="test.c"><
function><type><name>int</name></type> <name>main</name><parameter_list>()
</parameter_list> <block>{<block_content><expr_stmt><expr><call><name>
printf</name><argument_list>(<argument><expr><literal type="string">"Hello
World!"</literal></expr></argument>)</argument_list></call></expr>;</
expr_stmt><return>return <expr><literal type="number">0</literal></expr>;</
return></block_content>}</block></function></unit>
'''
example_code = '''
int main() {
    printf("Hello World!");
    return 0;
}
'''

[INST] You have know how to generate the code. Now, here\'s the example
code {example_code} and the srcml of that code {example}. can you follow
the example to generate the srcml of the {code} without explanation -- just
 srcml [\INST]
```

Template for AST generation after finetuning with code translation We used the following prompt for AST generation after finetuning with code translation, where {code} is the function for AST generation. {example_c- ode} is the source code example for generating AST and the {example} is the AST for the {example_code}.

```
example = '''
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit xmlns="http://
www.srcML.org/srcML/src" revision="1.0.0" language="C" filename="test.c"><
function><type><name>int</name></type> <name>main</name><parameter_list>()
</parameter_list> <block>{<block_content><expr_stmt><expr><call><name>
printf</name><argument_list>(<argument><expr><literal type="string">"Hello
World!"</literal></expr></argument>)</argument_list></call></expr>;</
expr_stmt><return>return <expr><literal type="number">0</literal></expr>;</
return></block_content>}</block></function></unit>
'''
example_code = '''
int main() {
    printf("Hello World!");
    return 0;
}'''

[INST] You have know how to translate the code. Now, here\'s the example
code {example_code} and the srcml of that code {example}. can you follow
the example to generate the srcml of the {code} without explanation -- just
 srcml [\INST]
```

Template for callgraph generation after finetuning with code summarization We used the following prompt for callgraph generation after finetuning with code summarization, where {code} is the function for callgraph generation and {method_name} is the name of the function.

```
[INST] The Java code is {code}. You have known how to do the code
summarization. Now, I want you to generate the callgraph path of the method
 {method_name} in the format of a->b->c etc, where -> means call and a is
the method. Please follow the format and rule and do not give me the
summary again. Remember the format of -> between each method name [\INST]
```

Template for callgraph generation after finetuning with code generation We used the following prompt for callgraph generation after finetuning with code generation, where {code} is the function for callgraph generation and {method_name} is the name of the function.

```
[INST]The Java code is {code}. You have known how to do the code generation
. Now, I want you to generate the callgraph path of the method {method_name
} in the format of a->b->c etc, where -> means call and a is the method.
Please follow the format and rule and do not give me the code again.
Remember the format of -> between each method name [\INST]
```

Template for callgraph generation after finetuning with code translation We used the following prompt for callgraph generation after finetuning with code translation, where {code} is the function for callgraph generation. {method_name} is the name of the function.

```
[INST]The Java code is {code}. You have known how to do the code
translation. Now, I want you to generate the callgraph path of the method {
method_name} in the format of a->b->c etc, where -> means call and a is the
 method. Please follow the format and rule and do not give me the code
again. Remember the format of -> between each method name [\INST]
```

Template for dataflow generation after finetuning with code summarization We used the following prompt for dataflow generation after finetuning with code summarization, where {example_data_flow} is the dataflow of the example code, {example_code} is the code for the example dataflow, {code} is the function for dataflow generation, {main_method_name} is the source of the dataflow, and {sink} is the sink of the dataflow.

```
example_code = '''
int main(int argc, char *argv[]) {
    if (argc > 1 && strcmp(argv[1], "42") == 0) {
        fprintf(stderr, "It depends!\n");
        exit(42);
    }
    printf("What is the meaning of life?\n");
    exit(0);
}
example_data_flow = '''
result: int main(int argc, char *argv[]) { -> if (argc > 1 && strcmp(argv
[1], "42") == 0) { '''

[INST] You have known how to do code summarization. Now, I do not want you
to generate the summary. Instead, I want you to generate the data flow of
the provided code. Here is one example that you can follow {example_code}
and its dataflow {example_data_flow}. could you please generate the data
flow of the C code {code} from source {main_method_name} to sink {sink} in
the format of a->b->c. Please follow the format and the example and do not
give me the description [\INST]
```

Template for dataflow generation after finetuning with code generation We used the following prompt for dataflow generation after finetuning with code generation, where {example_data_flow} is the dataflow of the example code, {example_code} is the code for the example dataflow, {code} is the function for dataflow generation, {main_method_name} is the source of the dataflow, and {sink} is the sink of the dataflow.

```
example_code = '''
int main(int argc, char *argv[]) {
    if (argc > 1 && strcmp(argv[1], "42") == 0) {
        fprintf(stderr, "It depends!\n");
        exit(42);
    }
    printf("What is the meaning of life?\n");
    exit(0);
}
example_data_flow = '''
result: int main(int argc, char *argv[]) { -> if (argc > 1 && strcmp(argv
[1], "42") == 0) { '''
[INST] You have known how to do code generation. Now, I do not want you to
generate the code. Instead, I want you to generate the data flow of the
provided code. Here is one example that you can follow {example_code} and
its dataflow {example_data_flow}. could you please generate the data flow
of the C code {code} from source {main_method_name} to sink {sink} in the
format of a->b->c. Please follow the format and the example and do not give
 me the description [\INST]
```

Template for dataflow generation after finetuning with code translation We used the following prompt for dataflow generation after finetuning with code translation, where {example_data_flow} is the dataflow of the example code, {example_code} is the code for the example dataflow, {code} is the function for dataflow generation, {main_method_name} is the source of the dataflow, and {sink} is the sink of the dataflow.

```
example_code = '''
int main(int argc, char *argv[]) {
    if (argc > 1 && strcmp(argv[1], "42") == 0) {
        fprintf(stderr, "It depends!\n");
        exit(42);
    }
    printf("What is the meaning of life?\n");
    exit(0);
}
example_data_flow = '''
result: int main(int argc, char *argv[]) { -> if (argc > 1 && strcmp(argv
[1], "42") == 0) {
'''

[INST] You have known how to do code translation. Now, I do not want you to
 translate the code. Instead, I want you to generate the data flow of the
provided code. Here is one example that you can follow {example_code} and
its dataflow {example_data_flow}. could you please generate the data flow
of the C code {code} from source {main_method_name} to sink {sink} in the
format of a->b->c. Please follow the format and the example and do not give
 me the description [\INST]
```

3.7 Research Methods – RQ1

Our research method for answering RQ1 is to evaluate closed-source LLMs on static analysis tasks without any finetuning. We compare the results with the tools that we describe in Section 3.3. Our goal is to understand model capabilities rather than optimizing performance details. To this end, we used the default parameter settings and simple prompts throughout the entire dataset although different parameters might have slightly different results. As an additional experiment, we followed Kojima et al. (2022) to add "Let's think step by step" at the end of our in-context learning prompt. Kojima et al. (2022) observed that we can simply add "Let's think step by step" at the end of the prompt to improve the models' performance on reasoning tasks.

For C/C++, we used the testset that we extracted to for evaluation. For Java, we used the testset proposed by LeClair and McMillan (2019) for evaluation. For srcML, we used Levenshtein based on Decker et al. (2020). We used Jaccard similarity as our evaluation metrics for callgraph and data flow graph analysis (Gharibi et al. 2018; Blokhin et al. 2013; Kargén and Shahmehri 2016). We also computed the pair accuracy and chain accuracy for both tasks.

3.8 Research Methods – RQ2

Our research method for answering RQ2 is to finetune the open-source LLMs and evaluate those models on static analysis tasks. We show those configurations in Tables 1 and 2. For example, we show that we finetune `jam` to do ast generation and evaluate `jam` with the same task. We do not finetune `jam` for callgraph generation because of the context length limit (see Section 3.4). Note that we increase the number of tokens to 10k for callgraph generation because callgraph generation would require larger context size. We use the hyperparameters recommended by the papers accompanying each model, which we show in Table 3. Recall from Section 3.4 that we used a QLoRA process for CodeLlaMA due to its size, but a full finetune process for jam.

Table 1 Datasets and models for different tasks in C/C++ and Java

	Task	Language	Dataset		Models	
			Name	Source	JAM	CodeLlaMA
Development	Code Summarization	Java	170k	Su and McMillan (2024)	x	x
		C/C++	Liu et al. (2021)	Liu et al. (2021)		x
	Code Translation	Java	CodeXGLUE	Lu et al. (2021)	x	x
		C/C++	C to Rust	Yu (2023)		x
		Java	Codetransocean	Yan et al. (2023)		x
		Java	PLTranslationEmpirical	Pan et al. (2024)		x
	Code Generation	Java	CONCODE	Lu et al. (2021)	x	x
		C/C++	Xlcost	Zhu et al. (2022)		x
		C/C++	Code Contest	Li et al. (2022)		x
Static Analysis	AST Generation	Java	funcom	LeClair and McMillan (2019)	x	x
		C/C++	Haque et al. (2021)	Haque et al. (2021)		x
	Call Graph Generation	Java	funcom	LeClair and McMillan (2019)	x	x
		C/C++	Haque et al. (2021)	Haque et al. (2021)		x
	Dataflow Graph Generation	Java	funcom	LeClair and McMillan (2019)	x	x
		C/C++	Haque et al. (2021)	Haque et al. (2021)		x

Table 2 Configurations of fine-tuning and evaluation of different models for each RQ. Note, jam is Java-only but for all other models we had separate Java and C fine-tuning and evaluation

		fine-tuning		evaluation
	model	static analysis	code dev.	
RQ2	jam	ast gen.	-	ast gen.
		dataflow gen.	-	dataflow gen.
	codellama	ast gen.	-	ast gen.
		call graph gen.	-	call graph gen.
		dataflow gen.	-	dataflow gen.
RQ3	jam	ast gen.	code sum.	code sum.
		dataflow gen.	code sum.	code sum.
		ast gen.	code gen.	code gen.
		dataflow gen.	code gen.	code gen.
	codellama	ast gen.	code sum.	code sum.
		call graph gen.	code sum.	code sum.
		dataflow gen.	code sum.	code sum.
		ast gen.	code trans.	code trans.
		call graph gen.	code trans.	code trans.
		dataflow gen.	code trans.	code trans.
		ast gen.	code gen.	code gen.
		call graph gen.	code gen.	code gen.
		dataflow gen.	code gen.	code gen.
RQ4	codellama	-	code sum.	ast gen.
		-	code sum.	callgraph gen.
		-	code sum.	dataflow gen.
		-	code trans.	ast gen.
		-	code trans.	callgraph gen.
		-	code trans.	dataflow gen.
		-	code gen.	ast gen.
		-	code gen.	callgraph gen.
		-	code gen.	dataflow gen.

Table 3 Fine-Tuning hyperparameters in our experiments

	jam	codellama
epoch	3	3
learning rate	3.00E-05	1.00E-04
batch size	4	2
gradient accum. steps	32	16
lora_r	-	64
lora_alpha	-	16
lora_dropout	-	0
quant_type	-	nf4
bits	-	4

3.9 Research Methods – RQ3

Our research method for answering RQ3 is to finetune the open-source LLMs on downstream tasks after those models have been finetuned on static analysis tasks. We show those configurations in Tables 1 and 2. For example, in the first row of jam column in RQ3, we show that we first finetune `jam` on the AST generation task. Then, we further finetuned it for

source code summarization. We compared this model with the one without finetuning with static analysis tasks to show whether static analysis improves the performance of LLMs on code intelligence tasks.

The first row of Table 1 show that the dataset we use for code summarization in Java is 170k dataset in Su and McMillan (2024). However, Table 1 also shows that we did not use jam for C/C++ datasets because `jam` is pretrained for Java only (see Section 3.4). Our guiding principle in all fine-tuning and evaluation is the same as RQ2. We used the base jam and CodeLlaMa models released by their respective papers where those papers pretrained using large datasets of code (see Section 3.4). Then, we further trained those models with the training sets which were released with each dataset (see Sections 3.2 and 3.3). We use the hyperparameters as described in RQ2.

We evaluated each configuration using the metrics and test datasets in the papers for each dataset. For code summarization, we used the metrics METEOR and USE as implemented by Su and McMillan (2024) and recommended by Haque et al. (2022). For code translation and code generation, we used Bleu score (Papineni et al. 2002) as in the dataset that we used for experiments (Iyer et al. 2018; Lu et al. 2021). We also have additional datasets for code generation and translation evaluated with pass@k Chen (2021) and compilation success rate.

3.10 Research Methods – RQ4

Our research method for answering RQ4 is to finetune models with code intelligence tasks and evaluate the models with static anlysis tasks. To this end, we used CodeLlama fintuned with each code intelligence to generate the three different static analysis tasks. We show the configuration in Table 2. For example, in the first row of RQ4, we show that we finetuned CodeLlama with code summarization. Then, we used the same model to generate AST. Note that we used simple prompts because our goal is to test the model's capability instead of finding the best results with better prompts.

3.11 Research Methods – RQ5

Our research method for answering RQ5 is to classify the types of error for static analysis tasks based on the current literature. To this end, we classify the error types for callgraph into four categories, which are missing direct call, extra direct calls, missing indirect calls, reference cycle, and predicted cycle based on Lehmann et al. (2023); Sui et al. (2020). Lehmann et al. (2023) constructed the callgraph that could potentially include cycles, direct edges, and indirect edges. We further classify into extra/missing direct calls and missing indirect calls based on false positive and false negative defined by Murphy et al. (1998), where false positive means additional edges and false negative means missing edges. For dataflow analysis, we followed Weideman et al. (2025) to categorize the errors into direct edges and indirect edges. Based on false positive and false negative, we further classified these two edges into extra direct edges, extra indirect edges, missing direct edges, and missing indirect edges. We categorized the AST errors into missing nodes and extra nodes based on AST difference discussed by Falleri et al. (2014); Fan et al. (2021). In addition, Alikhanifard and Tsantalis (2025) showed that AST can be incorrect semantically, so we added tag mismatch

into our categories. We also added parsing error because a generated AST may have an incorrect format/structure that causes the compile errors.

3.12 Threats to Validity

We organize threats to validity into threats to internal validity, external validity, and construct validity based on Wohlin et al. (2012). The threats to internal validity are that different hyperparamters can have different results. We mitigate this threat by following the accompanying papers. The threats to construct validity are that our experimental results may not be able to generalize to different tasks and programming languages. We mitigate this by using three different code development tasks and static analysis tasks along with two different programming languages although more programming languages can have better generalizability of the findings. We leave the experiments for other programming languages as future work. The threats to external validity are that different models might generate different results. We mitigate this threat by using two different commercial LLMs and two different open-source LLMs.

3.13 Limitations

Our prompt design is the key limitation in this paper. We used the simple prompt to test the reasoning capabilities of LLMs. The results show that LLMs do not do well on static analysis tasks. However, these results may not be able to generalize to more advanced prompting strategies (e.g., agentic development) and reasoning models, such as GPT-o series. Note that we did not have access to GPT-o series models at the time we conducted this experiment although Xie et al. (2025) show that more advanced models still struggle with tasks that require deeper semantic understanding.

4 Experimental Results

In this section, we discuss our experimental results to each of our RQs.

4.1 RQ1: Closed Models, Static Analysis Tasks

We found that in-context learning improves AST generation tasks and popular programming languages have higher improvement. We showed the results for AST generation on both Java and C/C++ in Table 4. In Java, we found that `gpt-base` has the Levenshtein score of 0.45 and `gemini-base` has the Levenshtein score of 0.8. We also found the similar results in `gemini` although we found higher results in `gemini`, which has Levenshtein score of 0.64 for `gemini-base` and Levenshtein score of 0.93 for `gemini-in-context`. For C/C++, we observed the similar trend as in Java. However, we observed the smaller improvement in C/C++ dataset. For example, we found around 80% improvement between GPT-base and GPT-in-context in Java dataset. However, we only found around 30% improvement between GPT-base and GPT-in-context in C/C++ dataset, which has around 50% difference in improvement. This result aligns with the finding that LLMs can have better performance on more popular programming languages (Jiménez 2024). The gap

Table 4 Results for srcml; gpt-base means the result without in-context learning; gpt-in-context means the result with in-context learning; codellama-finetuned means the result after finetuned

			Metrics
	task	model	Levenshtein
Java	srcml	gpt-base	0.45
		gpt-in-context	0.80
		gemini-base	0.64
		gemini-in-context	0.93
		codellama-finetuned	0.77
		jam-finetuned	0.89
C	srcml	gpt-base	0.46
		gpt-in-context	0.60
		gemini-base	0.65
		gemini-in-context	0.85
		codellama-finetuned	0.61
		jam-finetuned	-

between C/C++ and Java may imply the lack of reasoning skills on LLMs to generalize from one programming language to another programming language.

We showed an example from GPT in Fig. 1 for Java. We found that without in-context learning, the model tends to start the AST with `<class> <name> ClassName</name>` even if we only use a single method as we showed in Area 1 in Fig. 1c. With in-context learning, the method can correctly generate the AST that starts with `<function>` as shown in Area 1 in Fig. 1d. A possible explanation is that LLMs have been trained on huge dataset that has complete class, so those models assume all Java methods are inside a Java class. It is also possible that LLMs just try to learn the correct tags instead of correct syntax as in Area 2 in Fig. 1d and b.

We found that LLMs do a poor job on callgraph and dataflow graph generation and in-context learning does not improve the results. We showed the results for callgraph and dataflow graph in Table 5 for both Java and C/C++. For callgraph generation in Java, we found that `gpt-base` has Jaccard similarity of 0.121 and `gpt-in-context` has Jaccard similarity of 0.122. `gemini-base` and `gemini-in-context` have slightly higher performance, which is 0.186 and 0.191 respectively. We also found that chain accuracy is the lowest among all other metrics. In C/C++, we found the similar trend that in-context learning does not help to improve the results and chain accuracy is the lowest among all other metrics. For data flow generation, we found the similar results on both Java and C/C++ dataset. This result indicates that LLMs do not do well on callgraph and dataflow graph generation.

We found that reasoning prompts do not improve the performance for both callgraph and dataflow graph generation. We show the results for reasoning prompts in Table 5. For example, we observe in Java that `gpt-in-context` has Jaccard similarity of 0.122, but we only observe 0.113 in `gpt-in-context-rea-son`. The possible explanations are 1) LLMs do not have the fundamental ability for callgraph and dataflow generation, 2) in-context learning is only helping models to learn the format instead of the knowledge to do a task (Long et al. 2024), and 3) models still struggle to comprehend longer prompt although LLMs can take very long context (Zamfirescu-Pereira et al. 2023; Liu et al. 2024d). A possible improvement would be guiding LLMs with more detailed instructions on each task instead of providing examples.

Fig. 1 Example of SrcML generated by GPT and tool for Java

```
private TreeItem findNodeWithIdRecursive(TreeItem[] arrItem, String id)
{
   for(TreeItem current: arrItem)
   {
      String idInNode = (String) current.getData();
   }

}
```

(a) Source code

```
1
<function><type><specifier>private</specifier> <name>TreeItem</name></type>
<name>findNodeWithIdRecursive</name><parameter_list>(<parameter><decl><ty
pe><name><name>TreeItem</name><index>[]</index></name></type><name>arrI
tem</name></decl></parameter>,<parameter><decl><type><name>String</name>
</type><name>id</name></decl></parameter>)</parameter_list><block>{<block_c
ontent><for>for<control>(<init><decl><type><name>TreeItem</name></type>
<name>current</name><range>: 2
<expr><name>arrItem</name></expr></range></decl></init>)</control>
 <block>{<block_content> <decl_stmt><decl><type><name>String</name></type>
<name>idInNode</name>
<init>=<expr><operator>(</operator><name>String</name><operator>)</operator>
<call><name><name>current</name><operator>.</operator><name>getData</na
me></name><argument_list>()</argument_list></call></expr></init></decl>;</decl_
stmt>
</block_content>}</block></for>
</block_content>}</block></function>
```

(b) SrcML generated by tool

```
1
<class><name>ClassName</name>
<function><name>findNodeWithIdRecursive</name><parameter><type>
TreeItem[]</type><name>arrItem</name></parameter><parameter><type
>String</type><name>id</name></parameter><block><for><init>
<decl><type>TreeItem</type><name>current</name></decl><in><name>
arrItem</name></in></init><statement><assign><name>idInNode</name
><call><function>getData</function><on><name>current</name></on></
call>
</assign></block></function></class>
```

(c) SrcML generated by GPT without in-context learning

```
1
<function><type><specifier>private</specifier><name>TreeItem</name></type>
<name>findNodeWithIdRecursive</name><parameter_list>(<parameter><decl>
<type><name>TreeItem</name><index>[]</index></name></type><name>arrIte
m</name></decl></parameter>,<parameter><decl><type><name>String</name
></type> <name>id</name></decl></parameter>)</parameter_list>
<block>{<block_content><for_stmt><for_init><variable_decl><type><name>Tree
Item</name></type><name>current</name></variable_decl></for_init>
<for_each>
<in><name>arrItem</name></in></for_each><block>{<block_content><variable
_decl><type><name>String</name></type><name>idInNode</name></variable_
decl><assign><name>idInNode</name><operator>=</operator>(<call><name><
name>current</name><operator>.</operator><name>getData</name></name>
<argument_list>()</argument_list></call></name>)</assign>;
</block_content></block></for_stmt>
  <return><name>null</name></return>;
}</block_content></block></function>
```

(d) SrcML generated by GPT with in-context learning

Overall, we found that closed-source LLMs do not do static analysis very well. To our surprise, we found that in-context learning only helps to improve AST generation but not callgraph generation and dataflow generation. The possible explanation is that it is easy for LLMs to learn the tags for AST generation. However, LLMs struggle with looking at a

Table 5 Results for callgraph and dataflow graph analysis

			Metrics		
	task	model	Jaccard similarity	pair accuracy	chain accuracy
Java	callgraph	gpt-base	0.121	0.123	1.10E-04
		gpt-in-context	0.122	0.123	6.45E-06
		gpt-in-context-reason	0.113	0.114	1.54E-05
		gemini-base	0.186	0.190	1.90E-04
		gemini-in-context	0.191	0.195	1.20E-04
		gemini-in-context-reason	0.176	0.180	1.47E-07
		codellama-finetuned	0.910	0.940	2.20E-01
	dataflow graph	gpt-base	0.0047	0.0111	0.0
		gpt-in-context	0.0494	0.0951	5.37E-03
		gpt-in-context-reason	0.0479	0.0941	4.53E-03
		gemini-base	0.0045	0.0114	0.0
		gemini-in-context	0.0522	0.1028	2.31E-02
		gemini-in-context-reason	0.0711	0.1426	3.58E-03
		codellama-finetuned	0.54	0.74	2.03E-01
		jam-finetuned	0.38	0.54	0.18
C	callgraph	gpt-base	0.160	0.200	8.00E-06
		gpt-in-context	0.217	0.280	0
		gpt-in-context-reason	0.201	0.259	1.69E-30
		gemini-base	0.180	0.210	6.47E-05
		gemini-in-context	0.265	0.312	3.56E-05
		gemini-in-context-reason	0.263	0.368	1.20E-04
		codellama-finetuned	0.310	0.550	1.23E-04
	dataflow graph	gpt-base	0.0017	0.0054	1.12E-04
		gpt-in-context	0.0059	0.0126	5.13E-03
		gpt-in-context-reason	0.0047	0.0118	3.91E-03
		gemini-base	0.0022	0.0056	1.59E-03
		gemini-in-context	0.0093	0.0176	9.62E-03
		gemini-in-context-reason	0.0047	0.0118	3.91E-03
		codellama-finetuned	0.12	0.25	1.02E-01
		jam-finetuned	-	-	-

larger context for callgraph generation and more complicated reasoning tasks. These results suggest that LLMs do not have the same reasoning skills as human programmers.

4.2 RQ2: Open Models, Static Analysis Tasks

We observed that LLMs learn meaningful information after finetuning. Table 4 shows the results for AST generation in Java and C/C++. In Java dataset, we observed that open-source models `jam` reached Levenshtein score of 0.89 and CodeLlaMA reached Levenshtein score of 0.77 for AST generation. We also found that the result of CodeLlaMA is slightly lower. The possible explanation is that we used less data to finetune the model due to the computational cost although CodeLlaMA is already a larger model with more pretrained data compared with `jam`. To our surprise, we found that `jam` even outperforms `gpt-in-context` in this task in the simple prompt settings. This shows that an appropriate finetuning helps to improve this task. For C/C++, we also observed the lower Levenshtein score compared

with Java. Interestingly, we observed that LLMs can make a very simple syntactic error. For example, we observed in Fig. 2b in Area 1 that the AST generated by CodeLlaMA misses one ")" compared with Area 1 in Fig. 2c. Moreover, we observed in Fig. 2b in Area 2 that LLMs might continue generating the token at the end. This shows the lack of semantic understanding in LLMs.

We observed that models can only partially generate the correct callgraph. We showed the result for both C/C++ and Java for callgraph generation in Table 5. We found that CodeLlaMA has high Jaccard similarity score and pair accuracy for callgraph generation after finetuning. However, we found that the chain accuracy is still very low. We also found that C/C++ dataset has lower score compared with Java dataset. The possible explanation is that we input the entire file for C/C++ dataset and this makes the problem harder. In addition, we only use 10k tokens during prediction due to hardware limitation. It is also possible that using entire files would require more tokens. However, the conclusion is that lower chain accuracy and higher pair accuracy and Jaccrad similarity would remain the same based on our findings from Java. This indicates that LLMs struggle to look at the longer context although the input size is long (Li et al. 2024a).

We observed that the results of dataflow generation are similar to callgraph generation, i.e., low chain accuracy but higher Jaccard similarity and pair accuracy and C/C++ dataset

Fig. 2 Example of SrcML generated by CodeLlaMA and tool

```
NS_METHOD nsUnicodeToCP1125Constructor(nsISupports *aOuter,
REFNSIID aIID, void **aResult)
{
 return CreateTableEncoder(u1ByteCharset,
                (uMappingTable*) &g_ufMappingTable, 1,
                aOuter, aIID, aResult);
}
```

(a) Source code

```
<function><type><name>NS_METHOD</name></type>
<name>nsUnicodeToCP1125Constructor</name><parameter_list>(<parameter>
<decl><type><name>nsISupports</name><modifier>*</modifier></type><name>
aOuter</name></decl></parameter>,<parameter><decl><type><name>REFNSIID
</name></type><name>aIID</name></decl></parameter>,<parameter><decl>
<type><name>void</name><modifier>*</modifier><modifier>*</modifier></type>
<name>aResult</name></decl></parameter>)</parameter_list><block>{
<block_content><return>return<expr><call><name>CreateTableEncoder</name>
<argument_list>(<argument><expr><name>u1ByteCharset</name></expr>
</argument>, <argument><expr><operator>(</operator><name>uMappingTable
</name><operator>*</operator><operator>&amp;</operator><name> (1)
g_ufMappingTable</name><operator>)</operator></expr></argument>, <argument><expr><literal
type="number">1</literal></expr></argument>,
<argument><expr><name>aOuter</name></expr></argument>,<argument><expr>
<name>aIID</name></expr></argument>,<argument><expr><name>aResult</name></expr>
</argument>)</argument_list></call></expr>;</return></block_content>}
</block></function></unit>                                                    (2)
<?xml version="1.0" encoding="UTF- <?xmlversion="1.0" encoding="UTF-8" standalone="yes"?><unit
xmlns="http://www <?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit
xmlns="http://www.srcML.org/srcML/src" revision="
```

(b) SrcML generated by CodeLlaMA

```
<function><type><name>NS_METHOD</name></type>
<name>nsUnicodeToCP1125Constructor</name><parameter_list>(<parameter><decl><type>
<name>nsISupports</name>
<modifier>*</modifier></type><name>aOuter</name></decl></parameter>,
<parameter><decl><type><name>REFNSIID</name></type>
<name>aIID</name></decl></parameter>,<parameter><decl><type><name>void</name>
<modifier>*</modifier><modifier>*</modifier></type><name>aResult</name></decl></parameter>)
(1)</parameter_list><block>{<block_content><return>return<expr><call><name>CreateTableEncoder
</name><argument_list>(<argument><expr><name>u1ByteCharset</name></expr></argument>,
<argument><expr><operator>(</operator><name>uMappingTable</name><operator>*</operator>
<operator>)</operator><operator>&amp;</operator><name>g_ufMappingTable</name></expr>
</argument>, <argument><expr><literal type="number">1</literal></expr></argument>,
<argument><expr><name>aOuter</name></expr></argument>,
<argument><expr><name>aIID</name></expr></argument>,
<argument><expr><name>aResult</name></expr></argument>)</argument_list></call></expr>;
</return></block_content>}</block></function></unit>
```

(c) SrcML generated by tool

has lower results compared with Java. The results for both C/C++ and Java for dataflow graph generation are in Table 5. In Java, we also found that `jam` has very close results with CodeLlaMA on pair accuracy with the same finetuning data on data flow graph generation in Java dataset. This is to our surprise because larger models usually have bigger improvement in code development tasks, e.g., 25% improvement on meteor score for code summarization in Java. However, it only shows a slight improvement or even worse on static analysis tasks. This result implies that LLMs saturate faster in static analysis tasks compared with other code intelligence tasks. A possible explanation is that it is harder for LLMs to learn the meaningful information in static analysis tasks.

To sum up, we found the similar results on both open-source models and closed-source models. Specifically, we found that LLMs have larger improvement on AST generation after finetuning. However, LLMs struggle with callgraph generation and dataflow graph generation. This result shows LLMs' inability on static analysis tasks.

4.3 RQ3: Open Models, Static Analysis + Dev.

We do not strongly find the statistical difference between the models pretrained with static analysis and the baselines on both programming languages (Java and C/C++) and both models (CodeLlama and Jam) in code summarization. We showed the results for code summarization in Table 6 for Java and C/C++. The possible explanation is that the model looks at the keyword in the method for source code summarization instead of using static analysis tasks as what human programmers would do to comprehend source code. We showed an example in Java in Table 7. We found that LLMs tend to copy the technical jargon from the method for code summary. Specifically, we found that the summary from CodeLlaMA uses "CoordinateAxis" in the summary even after static analysis tasks. Similarly, we also observed the same phenomenon in C/C++. For example, in Table 8, we found that human reference uses "render" as a verb to describe the method. However, the summary from CodeLlaMA uses "show" as a verb, which matches the signature of the function. Interestingly, we found that some of the summaries are the same. This is because we use very small temperature, i.e., 1e-9 and models may generate the same result when the weights are similar (Peeperkorn

Table 6 Meteor and USE score for code summarization

			Metrics			
	model	config	METEOR	USE	p_m	p_u
Java	jam	base	40.95	68.80	-	-
		srcml	41.28	69.01	0.02	0.06
		dataflow graph	40.09	68.32	0.99	0.99
		callgraph	-	-	-	-
	codellama	base	51.44	74.56	-	-
		srcml	51.19	74.48	0.99	0.88
		dataflow graph	51.27	74.54	0.94	0.62
		callgraph	51.43	74.57	0.53	0.43
C	codellama	base	29.65	50.62	-	-
		srcml	29.96	50.85	0.99	0.99
		dataflow graph	29.87	50.73	0.99	0.93
		callgraph	29.97	50.91	0.99	0.99

Table 7 Code summarization example for Java

Method ID 29591858

```
static public String makeName( List axes) {
    StringBuffer buff = new StringBuffer();
    for (int i=0; i<axes.size(); i++) {
      CoordinateAxis axis = (CoordinateAxis) axes.get(i);
      if (i>0) buff.append("-");
      buff.append( axis.getName());
    }
    return buff.toString();
  }
```

codellama-base	takes a list of CoordinateAxis objects and returns a string that concatenates their names separated by hyphens
codellama-srcml	takes a list of CoordinateAxis objects and concatenates their names with hyphens in between, returning the resulting string
codellama-callgraph	takes a list of CoordinateAxis objects and concatenates their names with hyphens in between, returning the resulting string
codellama-dataflow	takes a list of CoordinateAxis objects and concatenates their names with hyphens in between, returning the resulting string
reference	takes a list of axes and returns a concatenated string of their names joined by "-"

Table 8 Code summarization example for C/C++

Method ID 120

```
static void knl_show_mc_route(u32 reg, char *s)
{
    int i;
    for (i = 0; i < KNL_MAX_CHANNELS; i++) {
        s[i*2] = knl_get_mc_route(i, reg) + '0';
        s[i*2+1] = '-';
    }
    s[KNL_MAX_CHANNELS*2 - 1] = '\0';
}
```

codellama-base	show the routing for a given MC channel
codellama-srcml	show the multicast route for a given register
codellama-callgraph	show the routing for a given MC channel
codellama-dataflow	show the routing for a given MC channel
reference	Render the MC_ROUTE register in human readable form

et al. 2024), which further showed that pretraining models with static analysis tasks does not improve code summarization tasks significantly.

In terms of BLEU and CodeBERT, we do not observe consistent improvement when we pretrain the models with static analysis tasks before finetuning models with code generation. Table 9 shows the result for code generation on both Java and C/C++. The possible explanation is that LLMs do not use static analysis tasks as the thought process. For example, in Table 10, we found that code generated by `jam-srcml` has better syntactic structure compared with `jam-base`. However, the underlying logic is still different from the refer-

Table 9 BLEU and CodeBERT for code generation

	model	config	BLEU	CodeBERT	p_b	p_c
Java	jam	base	23.51	0.95	-	-
		srcml	24.00	0.95	$p < 0.01$	$p < 0.01$
		dataflow graph	22.79	0.95	0.63	0.30
		callgraph	-	-	-	-
	codellama	base	31.42	0.97	-	-
		srcml	30.40	0.97	0.92	$p < 0.05$
		dataflow graph	34.22	0.97	$p < 0.001$	$p < 0.001$
		callgraph	28.93	0.97	0.99	0.62
C	codellama	base	48.20	0.93	-	-
		srcml	48.18	0.93	0.70	0.38
		dataflow graph	47.54	0.93	0.50	0.53
		callgraph	47.82	0.93	0.73	0.76

Table 10 Code generation example for Java

Method ID 988

```
boolean function(ProjCoordinate arg0, double arg1 ){
    double loc0 = arg0.x - this.x;
    if ( loc0 > arg1 ) { return false; }
    else { return true; }
}
```

jam-base

```
boolean function( double arg0){
return (( arg0 >= this.x ) && ( arg0
    <= this.x + this.z) && ( ( arg0
    >= this.y ) && ( arg0 <= this.
```

jam-srcml

```
boolean function(double arg0, double
    arg1 ){
return (arg0 >= x) && (arg0 <= arg1);
    }
```

jam-dataflow

```
boolean function(Coordinate arg0 ){
return ( x == arg0.x && y == arg0.y &&
     z == arg0.z );
}
```

ence. Another example in C/C++ dataset in Table 11 is that we observed that the code generated by CodeLlaMA without pretraining with any static analysis tasks has the logical error in the "toDecimal" method. After pretraining with static analysis tasks, we still found the same logical error. Interestingly, we observed that LLMs might generate code with incorrect details. In Table 12, we found that CodeLlaMA does not generate the correct parameters for Enneacontahexagon numbers. For human programmers to do this task, they would look at the correct parameters before writing code. This indicates that static analysis tasks help

Table 11 Code generation example for C/C++

Method ID 10

```
int binaryDec(int arr [], int n){
    int ans = 0;
    for(int i = 0 ; i < n ; i ++ )
        ans += arr [ i ] << ( n - i - 1 );
    return ans;
}
```

codellama-base	`int toDecimal(int arr[] , int n) { int res = 0 ; for(int i = 0; i < n;i ++){ res += arr [i] * ( 1 << i ); } return res; }`
codellama-srcml	`int toDecimal(int arr [], int n){ int res = 0; for(int i = 0; i < n; i ++ ) res += arr [ i ] * ( 1 << i ); return res; }`
codellama-dataflow	`int toDecimal(int arr [], int n){ int res = 0; for(int i = 0; i < n; i ++){ res += arr [i] * ( 1 << i ); } return res; }`
codellama-callgraph	`int convertFromBinary(int arr []){ int n = 0 , i = 0 ; while(arr[i] != 0 ){ n = n * 2 + arr [i]; i ++; } return n; }`

LLMs better understand syntactic but not semantic information. Similar results are observed by Ma et al. (2023).

For execution-based evaluation, we observed similar results as evaluated with BLEU score and CodeBERT in code generation. We show the result of the execution-based evaluation for code generation in Table 13. We found that static analysis tasks improve the compilation success rate, but we do not observe consistent improvement over pass@1. The possible explanation is that static analysis tasks improve the syntax of source code, but the underlying logic is still wrong. Note that the dataset that we used is extremely difficult as discussed in the original paper (Li et al. 2022), which requires a large amount of samples to

Table 12 Code generation example for C/C++

Method ID 10

```
int EnneacontahexagonNum(int n){
    return (94 * n * n - 92 * n) / 2;
}
```

codellama-base	`int enneacontahexagonNum(int n){ return ( 118 * n * n - 106 * n ) / 2; }`
codellama-srcml	`int enneacontahexagonNum(int n){ return ( 11 * n * n - 10 * n ) / 2; }`
codellama-dataflow	`int enneacontahexagonNum(int n){ return (11 * n * n - 10 * n) / 2; }`
codellama-callgraph	`int enneacontahexagonNum(int n){ return (14 * n * n - 14 * n + 1) / 2; }`

Table 13 Execution-based evaluation on code generation

	model	config	Pass@1	Compilation rate
C	codellama	base	0.024	0.45
		srcml	0.018	0.46
		dataflow graph	0.048	0.57
		callgraph	0.018	0.54

have a higher pass@k. Our goal in this paper is to understand the model capabilities instead of finding the optimal results. To this end, our goal is to ensure every model has the same settings instead of finding the optimal settings for each model.

For the code translation dataset, we observe statistical significant improvement in the BLEU score in the Java dataset after we pretrained models with callgraphs. However, we do not observe the same improvement in the C/C++ dataset. We show the results for both Java and C/C++ in Table 14. The possible explanation is that pretraining models with callgraphs help models look at the larger context and code translation would require a larger context for more accurate translation. However, we cannot strongly attribute the improvement to pretrainig models with callgraphs because we do not observe similar improvement in the C to Rust dataset. Interestingly, we found that code translation has higher performance compared with other tasks in Java datasets. It is also possible that Java and C# have very similar API calls and syntax, so LLMs can easily copy the information from the source language as we do not observe a similar trend in the C to Rust dataset.

Table 14 BLEU score for code translation

	model	config	BLEU	p
Java	jam	base	50.34	-
		srcml	49.26	0.90
		dataflow graph	50.25	0.08
		callgraph	-	-
	codellama	base	80.36	-
		srcml	80.52	0.56
		dataflow graph	79.82	0.57
		callgraph	81.84	$p < 0.01$
C	codellama	base	37.82	-
		srcml	37.50	0.84
		dataflow graph	41.64	0.91
		callgraph	39.70	0.91

Table 15 Execution-Based evaluation on code translation

	model	config	Pass@1	Compilation rate
Java	codellama	base	0.0	0.60
		srcml	0.0	0.51
		dataflow graph	0.0	0.52
		callgraph	0.0	0.69

Regarding execution-based metrics on code translation, we observed that callgraph improves the compilation success rate, but none of the static analysis tasks improves the pass@1. We show the results of the execution-based evaluation for code translation in Table 15. The possible explanation is that callgraph provides the more information between the function calls for code translation tasks. However, LLMs do not learn the semantic information during pretraining. This indicates the lack of semantic reasoning skills in LLMs. Note that our goal is to test the model's capabilities instead of finding the best performance, so we only used the simple prompts. The low pass@1 also aligns with the results in Pan et al. (2024) in llama-based models.

It is possible that we do not use proper prompts to guide the models to perform downstream tasks after pretraining. However, Lu et al. (2024b) observed that the emerging ability of LLMs comes from linguistic knowledge and model memory. To this end, we examine the performance of jam models more carefully because jam models require the specific prompt format instead of free-form, which reduces the threat of improper prompts. We show the results in Tables 6, 9, and 14. We do not observe strong improvement over three code intelligence tasks. Specifically, we do not observe any statistical difference for source code summarization and code generation. This shows that models do not use static analysis as the fundamental knowledge to perform code intelligence tasks as human programmers.

Overall, our results lead to two conclusions 1) LLMs might have an alien thought process for code comprehension compared with human programmers 2) the static analysis tasks improve the overall syntax of the code, but static analysis tasks do not improve the semantics.

4.4 RQ4: Open Models, Dev. + Static Analysis

We observed significantly lower results compared with models finetuned to do the tasks. We showed the results for AST generation in Table 16 and callgraph and dataflow graph generation in Table 17. This shows that LLMs do not use static analysis tasks for code intelligence tasks. For example, in Table 18, we observed that "setHorizontalAlignment" does not call "setVerticalAlignment", but CodeLlaMA generates a path where "setHorizontalAlignment" is calling "setVerticalAlignment". This shows that LLMs do not have the fundamental understanding of callgraph generation even though it has already learnt to generate code summarization. Another possible explanation is that LLMs could learn the form from the downstream tasks instead of the semantic meaning. For example, Bender and Koller (2020) showed that LLMs could learn both the "form" of the language and the "semantics" of the language. They observed that LLMs do not need to learn the semantics of the language to have good performance on code development tasks. LLMs can do the downstream tasks well while learning the form from the training data only. This result aligns with the finding that LLMs only memorize the form in the training data instead of understanding the semantics of the training data (Ma et al. 2025). This would guide future research to develop new model architectures or new approaches to train models.

Table 16 Results for srcml generated by models finetuned with code intelligence tasks

			Metrics
	task	model	Levenshtein
Java	srcml	codesum	0.350
		codegen	0.059
		codetrans	0.430
C	srcml	codesum	0.035
		codegen	0.133
		codetrans	0.243

Table 17 Results for callgraph and dataflow graph generated by the models finetuned with various code intelligence tasks

			Metrics		
	task	model	Jaccard similarity	pair accuracy	chain accuracy
Java	callgraph	codesum	0.061	0.076	0
		codegen	0.011	0.046	1.07E-5
		codetrans	0.013	0.050	0
	dataflow graph	codesum	0.0019	0.0038	4.96E-04
		codegen	0.0	0.0	0.0
		codetrans	0.0044	0.0072	1.44E-03
C	callgraph	codesum	0.002	0.025	0.00012
		codegen	0.048	0.075	2.64E-06
		codetrans	0.016	0.028	2.14E-07
	dataflow graph	codesum	0.0	0.0	0.0
		codegen	0.0	0.0	0.0
		codetrans	1.36E-06	4.91E-06	0.0

Table 18 Example that shows that LLMs still struggle with generating correct call path after we finetuned it with code summarization

```
public void setAlignment(int row, int column,
    HorizontalAlignmentConstant hAlign,  VerticalAlignmentConstant
        vAlign){
    prepareCell(row, column);
    setHorizontalAlignment(row, column, hAlign);
    setVerticalAlignment(row, column, vAlign);
}

public void setHorizontalAlignment(int row, int column,
    HorizontalAlignmentConstant align){
    prepareCell(row, column);
    DOM.setStyleAttribute(getCellElement(tableElem, row,  column),
        "textAlign", align.getTextAlignString());
}

protected void prepareCell(int row, int column){
    prepareRow(row);
    if (column < 0){
      throw new IndexOutOfBoundsException(
        "Cannot create a column with a negative index: " + column)
            ;
    }

    // Ensure that the requested column exists.
    int cellCount = getCellCount(row);
    for (int i = cellCount; i <= column; ++i)
      addCell(row);
}
```

reference	1. setAlignment -> prepareCell 2. setAlignment -> setHorizontalAlignment -> prepareCell
codellama	1. setAlignment -> prepareCell -> setHorizontalAlignment -> setVerticalAlignment -> setAttr

4.5 RQ5: Analysis of Error Types

We found that extra direct calls is the common error type followed by missing direct calls and cycles in Java dataset. In C dataset, we found that missing direct calls is the common error types followed by extra direct calls and missing indirect calls. We show the results for callgraph in Table 19. In the Java dataset, we observed that LLMs tend to predict the extra direct calls in callgraph. The possible explanation is that direct calls are more straightforward than indirect edges. This phenomenon can also be verified in the number of functions with cycles. For example, in Java, we only observed three functions with cycles in the reference callgraph. However, we observed 1,510 functions with cycles in the predicted callgraph in `gpt-base`. We observed that finetuning CodeLlaMA improves overall results and reduces the number of cycles. However, we do not observe the reasoning prompt helps to improve the results. Regarding C/C++ dataset, we observed that callgraph generated by

Table 19 Error types analysis for callgraph generation; MDC stands for missing direct call; EDC stands for extra direct call; MIC stands for missing indirect call; RC stands for reference cycles; PC stands for predicted cycles

			Metrics				
	task	model	MDC	MIC	RC	PC	EDC
Java	callgraph	gpt-base	10,246	68	3	1,510	11,735
		gpt-in-context	10,316	68	3	1,243	11,985
		gpt-in-context-reason	10,480	65	3	1,297	12,120
		gemini-base	10,480	65	3	1,297	12,120
		gemini-in-context	9,855	65	3	1,087	10,704
		gemini-in-context-reason	10,146	64	3	534	9,953
		codellama-finetuned	3,256	65	3	43	2,738
C	callgraph	gpt-base	21,773	10,863	82	4,075	20,016
		gpt-in-context	22,662	11,170	82	1,598	20,886
		gpt-in-context-reason	22,953	11,200	82	1,722	21,480
		gemini-base	22,589	11,049	82	1,130	20,430
		gemini-in-context	21,551	10,899	82	1,033	19,203
		gemini-in-context-reason	21,883	11,241	82	476	14,422
		codellama-finetuned	18,928	11,169	82	649	10,591

LLMs tends to miss more direct edges and finetuning CodeLlaMA improves the results. Note that it is possible that one function has extra direct edges and misses direct edges based on our calculation. We show an example in Table 20. When we compared path 1 in the reference and path 1 in the GPT predicted path, we have extra direct edges, i.e. from prepareCell to prepareRow, whereas we have one missing direct edge when we compared path 1 in the reference and path 4 in the GPT predicted path.

For dataflow analysis in Java, we found that extra direct edges is the most common error followed by extra indirect edges. In C dataset, we observed that extra direct edges is the most common error followed by missing direct edges. We showed the results for dataflow analysis in Table 21. The possible explanation is that LLMs are lack of semantic reasoning skills to generate the correct edges. We showed one example for C in Table 22. We found that the reference only has one path, whereas LLMs generate multiple paths and all of the paths are incorrect. This example would be counted in both extra direct edges and missing edges. This result aligns with the findings by Ma et al. (2023); Xie et al. (2025) that LLMs tend to hallucinate when doing more complex reasoning tasks.

We observed that tag mismatch is the most common error in LLMs generated AST and in-context learning reduces the number of functions with tag mismatches. We show the results for AST in Table 23. For example, we observed 7,135 functions have tag mismatches in `gpt-base`, whereas we only observed 703 functions that have tag mismatches in `gpt-in-context`. The possible explanations are 1) LLMs are lack of semantic reasoning skills, so LLMs cannot generate the proper tags based on the semantic information. and 2) LLMs do not know the proper SrcML format, so in-context learning helps LLMs to learn the proper format. For example, we observed that the tools use `decl` as the tag whereas LLMs use `expr` as the tag. To our surprise, we found that finetuning improves the Levenshtein, but we observe the lower compile success rate. The explanation is that we limit the output length of CodeLlama to 512 tokens due to computational cost. We showed one example in

Table 20 Example of error types analysis for callgraph

```
public void setAlignment(int row, int column,
    HorizontalAlignmentConstant hAlign,  VerticalAlignmentConstant
        vAlign){
    prepareCell(row, column);
    setHorizontalAlignment(row, column, hAlign);
    setVerticalAlignment(row, column, vAlign);
}

public void setHorizontalAlignment(int row, int column,
    HorizontalAlignmentConstant align){
    prepareCell(row, column);
    DOM.setStyleAttribute(getCellElement(tableElem, row,  column),
        "textAlign", align.getTextAlignString());
}

protected void prepareCell(int row, int column){
    prepareRow(row);
    if (column < 0){
      throw new IndexOutOfBoundsException(
        "Cannot create a column with a negative index: " + column)
            ;
    }

    // Ensure that the requested column exists.
    int cellCount = getCellCount(row);
    for (int i = cellCount; i <= column; ++i)
      addCell(row);
}
```

reference	1. setAlignment -> prepareCell 2. setAlignment -> setHorizontalAlignment -> prepareCell
gpt	1. setAlignment->prepareCell->prepareRow 2. setAlignment->prepareCell->getCellCount 3. setAlignment->prepareCell->addCell 4. setAlignment->setHorizontalAlignment 5. setAlignment->setVerticalAlignment

Table 24 that the SrcML is incomplete. Overall, our results align with our previous findings that LLMs are lack of semantic reasoning skills.

4.6 Discussion

The key novelty of this paper is that we explore whether LLMs use static analysis to do code development tasks like a human programmer would. Recent research has shown that pre-training LLMs with large source code and increasing model size improve code development tasks (Wang et al. 2021; Roziere et al. 2023; Li et al. 2023b). However, research has shown that programmers build mental models of program behavior through static analysis (Jiang et al. 2016; Beller et al. 2016; Vassallo et al. 2020; Wallace et al. 2025). One would expect that pretraining LLMs with static analysis tasks would also improve the code development

Table 21 Error types analysis for dataflow generation; EDE stands for extra direct edges; MDE stands for missing direct edges; MIE stands for missing indirect edges; EIE stands for extra indirect edges

			Metrics			
	task	model	EDE	MDE	MIE	EIE
Java	dataflow	gpt-base	7,042	7,041	5,602	7,042
		gpt-in-context	6,997	6,436	4,501	6,934
		gpt-in-context-reason	7,002	6,487	4,502	6,940
		gemini-base	7,036	7,033	5,602	7,035
		gemini-in-context	6,809	6,399	4,711	6,554
		gemini-in-context-reason	6,688	6,284	4,578	6,200
		codellama-finetuned	6,119	5,442	4,756	6,087
C	dataflow	gpt-base	8,128	8,104	5,439	8,128
		gpt-in-context	7,957	8,042	5,456	7,922
		gpt-in-context-reason	7,964	8,053	5,458	7,951
		gemini-base	8,093	8,084	5,445	8,092
		gemini-in-context	7,763	7,960	5,397	7,700
		gemini-in-context-reason	7,962	7,978	5,403	7,902
		codellama-finetuned	5,770	6,497	5,046	5,578

Table 22 Example that shows the common errors in Dataflow graph generation. The sink is PR_Unlock and the source is from the function signature

```
void jsd_Unlock(JSDStaticLock* lock){
    void* me;
    ASSERT_VALID_LOCK(lock);
    _CURRENT_THREAD(me);
    JS_ASSERT(lock->owner == me);
    if(lock->owner != me)
        return;
    if(--lock->count == 0)
    {
        lock->owner = NULL;
        PR_Unlock(lock->lock);
    }
}
```

reference	`jsd_Unlock(JSDStaticLock* lock) => ASSERT_VALID_LOCK(lock); => PR_Unlock(lock->lock);`
codellama	1. `_CURRENT_THREAD(me) => JS_ASSERT(lock->owner == me)` 2. `JS_ASSERT(lock->owner == me) => if(lock->owner != me)` 3. `if(lock->owner != me) => return` 4. `if(--lock->count == 0) => lock->owner = NULL` 5. `lock->owner = NULL => PR_Unlock(lock->lock)` 6. `if(--lock->count == 0) => PR_Unlock(lock->lock)`

Table 23 Error types analysis for AST generation; MN stands for missing nodes; EN stands for extra nodes; TM stands for tag mismatches; PE stands for parsing errors

			Metrics			
	task	model	MN	EN	TM	PE
Java	srcml	gpt-base	5,380	2,428	7,135	1,498
		gpt-in-context	971	1,036	703	7,249
		gemini-base	0	0	0	8,625
		gemini-in-context	1,412	4,253	1,541	3,876
		codellama-finetuned	21	10	5	8,282
C	srcml	gpt-base	4,216	3,398	5,988	3,567
		gpt-in-context	7	11	4	9,549
		gemini-base	2,719	5,824	6,313	2,690
		gemini-in-context	2,437	5,432	3,941	3,285
		codellama-finetuned	2	3	2	9,549

Table 24 Example of error types analysis for AST generation in CodeLlama

<?xml version="1.0" encoding="UTF-8" standalone="yes"?><unit xmlns="http://www.srcML.org/srcML/src" evision="1.0.0" language="Java" filename="temp.java"> <function><type><specifier>private</specifier> < name>TreeItem</name></type> <name>findNodeWithIdRecursive</name><parameter_list>(<parameter><decl>< type><name><name>TreeItem</name><index>[]</index></name></type> <name>arrItem</name></decl></parameter >, <parameter><decl><type><name>String</name></type> <name>id</name></decl></parameter>)</ parameter_list><block>{<block_content><for>for<control>(<init><decl><type><name>TreeItem</name></type> <name>current</name><range>: <expr><name><name>arrItem</name><index>[]</index></name></expr></range ></decl></init>)</control><block>{<block_content><decl_stmt><decl><type><name>String</name></type> < name>idInNode</name> <init>= <expr><operator>(</operator><name>String</name><operator>)</operator> < call><name><name>current</name><operator>.</operator><name>getData</name></name><argument_list>()</ argument_list></call></expr></init></decl>;</decl_stmt><if_stmt><if>if<condition>(<expr><call><name>< name>idInNode</name><operator>.</operator><name>equals</name></name><argument_list>(<argument><expr>< name>id</name></expr></argument>)</argument_list></call></expr>)</condition><block type="pseudo"><> block_content><return>return <expr><name>current</name></expr>;</return></block_content></block></if ></if_stmt><decl_stmt><decl><type><name>TreeItem</name></type> <name>itemInside</name> <init>= <

tasks if LLMs use the same thought process as human programmers. One would also expect LLMs have static analysis as a byproduct if LLMs use static analysis while doing code development tasks. Therefore, we conducted exhaustive experiments to answer this question. Although several papers have more thorough discussion and define more thorough taxonomies on the error types of the tasks (Sun et al. 2024b; Pan et al. 2024; Yuan et al. 2023; Jiang et al. 2024a; Ma et al. 2023), those papers mainly focus on the discussion of individual tasks without building the connection between these two tasks. Most of the current papers that discuss the connection between these two tasks only focus on how to include static analysis in the models (Alon et al. 2019a; Sun et al. 2020; LeClair et al. 2020; Sun et al. 2020; Tang et al. 2022; Bansal et al. 2023; Du and Yu 2023) and in the prompts (Wang et al. 2024a; Su et al. 2025; Luo 2025; Zhang et al. 2026) to improve code development tasks. Providing static analysis in the input would give models opportunities to learn the "form" of the programming languages instead of understanding whether models use the "semantics" of static analysis. Therefore, we explore this question in this paper.

We found that LLMs do not do static analysis. Specifically, we have two key observations. First, training models with static analysis tasks does not improve the code development tasks and training models with code development tasks does not lead to models being able to perform static analysis tasks. This would imply that LLMs do not use static analysis tasks while doing code development tasks and LLMs do not have an understanding of the semantics of static analysis. Second, LLMs could make the same logical mistakes as in the

models without being pretrained with static analysis tasks. This shows that LLMs learn the "form" instead of "semantics". This finding suggests future research developing new model architectures and training procedures that mimic what human programmers would do for software engineering tasks.

5 Conclusion

In this paper, we have four key observations. First, we found that in-context learning and finetuning only improve the performance on AST generation. However, in-context learning does not improve more complex static analysis tasks and finetuning only improves the more complex static analysis tasks partially. Specifically, we do not observe any significant improvement on callgraph generation and dataflow graph generation with in-context learning. Moreover, we found that finetuning improves pair accuracy and Jaccard similarity although we do not observe a significant improvement on chain accuracy. This result shows that LLMs struggle with more complex reasoning.

Second, we found that pretraining models with various static analysis tasks do not necessarily improve the code development tasks. Specifically, we do not strongly observe the statistical difference across all datasets and all programming languages when we pretrain models with different static analysis tasks. Another piece of evidence that shows that static analysis tasks do not improve code development tasks is that LLMs tend to make the same logical errors after pretraining models with static analysis tasks. This shows that LLMs do not use static analysis as human programmers would.

Third, we found that LLMs do not have static analysis as a byproduct when we train LLMs with code development tasks. We observed significantly lower results compared with models specifically finetuned or prompted to do the tasks. The possible explanation is that LLMs learn the form of the training data instead of the semantics of the training data. In other words, LLMs could simply memorize the input form instead of understanding the semantic information of the input.

Finally, we observed that LLMs tend to generate outputs with simple semantic relations. For example, we found that LLMs tend to predict more cycles than it actually has in the callgraph. The possible explanation is that cycles are more straightforward for LLMs to predict than predicting indirect edges. This would further show that LLMs are lack of semantic reasoning skills and guide future research to develop new model architectures and training procedures to help LLMs to learn the semantics of inputs. Overall, we found that LLMs do not use static analysis like a human would.

Acknowledgements This work is supported in part by the NSF grants CCF-2100035, CCF-2211428, and CCF-2211429. Any opinions, findings, and conclusions expressed herein are the authors' and do not necessarily reflect those of the sponsors.

Author Contributions Chia-Yi Su designed the experiments, implemented the code for the experiments, and wrote the manuscript. Collin McMillan wrote the manuscript and designed the experiments.

Code / Data Availability We release all data, scripts, survey materials, and results at https://github.com/apcl-research/llm-reason.

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

Ethical Approval This study was conducted in accordance with institutional guidelines and approved by the Institutional Review Board.

Informed Consent Not applicable.

Clinical Trial Number Not applicable.

References

Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S et al (2023) Gpt-4 technical report. arXiv preprint arXiv:2303.08774

Agrahari V, Chimalakonda S (2020) Ast[ar] - towards using augmented reality and abstract syntax trees for teaching data structures to novice programmers. In: 2020 IEEE 20th international conference on advanced learning technologies (ICALT), pp 311–315. https://doi.org/10.1109/ICALT49669.2020.00100

Alanazi R (2021) Software Analytics for Improving Program Comprehension. University of Missouri-Kansas City

Alanazi R, Gharibi G, Lee Y (2021) Facilitating program comprehension with call graph multilevel hierarchical abstractions. J Syst Softw 176:110945

Alikhanifard P, Tsantalis N (2025) A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. ACM Trans Softw Eng Methodol 34(2). https://doi.org/10.1145/3696002

Allamanis M (2019) The adverse effects of code duplication in machine learning models of code. In: Proceedings of the 2019 ACM SIGPLAN international symposium on new ideas, new paradigms, and reflections on programming and software, pp 143–153

Alon U, Brody S, Levy O, Yahav E (2019a) code2seq: Generating sequences from structured representations of code. International conference on learning representations

Alon U, Zilberstein M, Levy O, Yahav E (2019b) code2vec: Learning distributed representations of code. Proc ACM Program Lang 3(POPL):1–29

Al-Saiyd NA (2017) Source code comprehension analysis in software maintenance. In: 2017 2nd International conference on computer and communication systems (ICCCS), pp 1–5. https://doi.org/10.1109/CCOMS.2017.8075175

Anil R, Dai AM, Firat O, Johnson M, Lepikhin D, Passos A, Shakeri S, Taropa E, Bailey P, Chen Z et al (2023) Palm 2 technical report. arXiv preprint arXiv:2305.10403

arstechnica (2024) Google ceo says over 25% of new google code is generated by ai. https://arstechnica.com/ai/2024/10/google-ceo-says-over-25-of-new-google-code-is-generated-by-ai/

Ayewah N, Pugh W, Hovemeyer D, Morgenthaler JD, Penix J (2008) Using static analysis to find bugs. IEEE Softw 25(5):22–29. https://doi.org/10.1109/MS.2008.130

Bachmann M (2020) Rapidfuzz: Rapid fuzzy string matching in python. https://github.com/rapidfuzz/RapidFuzz, accessed: 14 Nov 2025

Balloccu S, Schmidtová P, Lango M, Dušek O (2024) Leak, cheat, repeat: Data contamination and evaluation malpractices in closed-source llms. arXiv preprint arXiv:2402.03927

Banerjee S, Lavie A (2005) Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In: Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization, pp 65–72

Bansal A, Eberhart Z, Karas Z, Huang Y, McMillan C (2023) Function call graph context encoding for neural source code summarization. IEEE Trans Software Eng 49(9):4268–4281. https://doi.org/10.1109/TSE.2023.3279774

Bansal A, Wallace R, Karas Z, Tang N, Huang Y, Li TJJ, McMillan C (2024) Programmer visual attention during context-aware code summarization. arXiv preprint arXiv:2405.18573

Bansal A, Haque S, McMillan C (2021) Project-level encoding for neural source code summarization of subroutines. International conference on program comprehension

Barke S, James MB, Polikarpova N (2023) Grounded copilot: How programmers interact with code-generating models. Proc ACM Program Lang 7(OOPSLA1). https://doi.org/10.1145/3586030
Becker BA, Denny P, Finnie-Ansley J, Luxton-Reilly A, Prather J, Santos EA (2023) Programming is hard - or at least it used to be: Educational opportunities and challenges of ai code generation. In: Proceedings of the 54th ACM technical symposium on computer science education, vol. 1, association for computing machinery, SIGCSE 2023, pp 500–506. https://doi.org/10.1145/3545945.3569759
Beller M, Bholanath R, McIntosh S, Zaidman A (2016) Analyzing the state of static analysis: A large-scale evaluation in open source software. In: 2016 IEEE 23rd International conference on software analysis, evolution, and reengineering (SANER), vol 1, pp 470–481. https://doi.org/10.1109/SANER.2016.105
Bender EM, Koller A (2020) Climbing towards nlu: On meaning, form, and understanding in the age of data. In: Proceedings of the 58th annual meeting of the association for computational linguistics, pp 5185–5198
Bergeretti JF, Carré BA (1985) Information-flow and data-flow analysis of while-programs. ACM Trans Program Lang Syst 7(1):37–61. https://doi.org/10.1145/2363.2366
Blokhin K, Saxe J, Mentis D (2013) Malware similarity identification using call graph based system call subsequence features. In: 2013 IEEE 33rd international conference on distributed computing systems workshops, pp 6–10, https://doi.org/10.1109/ICDCSW.2013.55
Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
Chen M (2021) Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374
Chen J, Mueller J (2024) Automated data curation for robust language model fine-tuning. arXiv preprint arXiv:2403.12776
Chen W, Ma X, Wang X, Cohen WW (2022) Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. arXiv preprint arXiv:2211.12588
Chen J, Pan Z, Hu X, Li Z, Li G, Xia X (2024) Evaluating large language models with runtime behavior of program execution. arXiv preprint arXiv:2403.16437
Chen X, Gao C, Chen C, Zhang G, Liu Y (2025) An empirical study on challenges for llm application developers. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3715007
Decker MJ, Collard ML, Volkert LG, Maletic JI (2020) srcdiff: A syntactic differencing approach to improve the understandability of deltas. J Softw Evol Process 32(4):e2226
Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L (2024) Qlora: Efficient finetuning of quantized llms. Adv Neural Inf Process Syst 36
doxygen (2023) Doxygen. https://www.doxygen.nl/
Du Y, Yu Z (2023) Pre-training code representation with semantic flow graph for effective bug localization. In: Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering, pp 579–591
Egan MH, McDonald C (2014) Program visualization and explanation for novice c programmers. Proc Sixteenth Austral Comput Educ Conf Vol 148:51–57
Eisenbarth T, Koschke R, Simon D (2001) Aiding program comprehension by static and dynamic feature analysis. In: Proceedings IEEE international conference on software maintenance. ICSM 2001, pp 602–611, https://doi.org/10.1109/ICSM.2001.972777
Falleri JR, Morandat F, Blanc X, Martinez M, Monperrus M (2014) Fine-grained and accurate source code differencing. In: Proceedings of the 29th ACM/IEEE international conference on automated software engineering, association for computing machinery, ASE '14, pp 313–324. https://doi.org/10.1145/2642937.2642982, https://doi-org.proxy.library.nd.edu/10.1145/2642937.2642982
Fan Y, Xia X, Lo D, Hassan AE, Wang Y, Li S (2021) A differential testing approach for evaluating abstract syntax tree mapping algorithms. In: Proceedings of the 43rd international conference on software engineering, IEEE Press, ICSE '21, pp 1174–1185, https://doi.org/10.1109/ICSE43902.2021.00108
Fan L, Liu J, Liu Z, Lo D, Xia X, Li S (2024) Exploring the capabilities of llms for code change related tasks. ACM Trans Softw Eng Methodol
Feautrier P (1991) Dataflow analysis of array and scalar references. Int J Parallel Prog 20:23–53
Gharibi G, Tripathi R, Lee Y (2018) Code2graph: automatic generation of static call graphs for python source code. In: Proceedings of the 33rd ACM/IEEE international conference on automated software engineering, association for computing machinery, ASE '18, pp 880–883. https://doi.org/10.1145/3238147.3240484
Gu Q (2023) Llm-based code generation method for golang compiler testing. In: Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering, pp 2201–2203
Haiduc S, Aponte J, Moreno L, Marcus A (2010) On the use of automated text summarization techniques for summarizing source code. In: 2010 17th Working conference on reverse engineering, pp 35–44. https://doi.org/10.1109/WCRE.2010.13

Haque S, Bansal A, Wu L, McMillan C (2021) Action word prediction for neural source code summarization. In 28th IEEE international conference on software analysis, evolution and reengineering

Haque S, Eberhart Z, Bansal A, McMillan C (2022) Semantic similarity metrics for evaluating source code summarization. In: Proceedings of the 30th IEEE/ACM international conference on program comprehension, association for computing machinery, ICPC '22, pp 36–47. https://doi.org/10.1145/3524610.3527909

Harth E, Dugerdil P (2017) Program understanding models: An historical overview and a classification. In: Proceedings of the 12th international conference on software technologies, 24-26 July 2017

He F, Zhu T, Ye D, Liu B, Zhou W, Yu PS (2024) The emerged security and privacy of llm agent: A survey with case studies. arXiv preprint arXiv:2407.19354

He J, Treude C, Lo D (2025) Llm-based multi-agent systems for software engineering: Literature review, vision and the road ahead. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3712003

Holt AW, Turanski WJ (1960) Man-to-machine communication and automatic code translation. In: Papers presented at the May 3-5, 1960, Western joint IRE-AIEE-ACM computer conference, association for computing machinery, IRE-AIEE-ACM '60 (Western), pp 329–339. https://doi.org/10.1145/1460361.1460405

Hong J, Ryu S (2024) Don't write, but return: Replacing output parameters with algebraic data types in c-to-rust translation. Proc ACM Program Lang 8(PLDI). https://doi.org/10.1145/3656406

Hovemeyer D, Hellas A, Petersen A, Spacco J (2016) Control-flow-only abstract syntax trees for analyzing students' programming progress. In: Proceedings of the 2016 ACM conference on international computing education research, pp 63–72

Iyer S, Konstas I, Cheung A, Zettlemoyer L (2018) Mapping language to code in programmatic context. arXiv preprint arXiv:1808.09588

Jain R, Purandare R (2025) Assessing large language models in comprehending and verifying concurrent programs across memory models. arXiv preprint arXiv:2501.14326

Jiang S, McMillan C, Santelices R (2016) Do programmers do change impact analysis in debugging? Empir Softw Eng 1–39. https://doi.org/10.1007/s10664-016-9441-9

Jiang S, McMillan C, Santelices R (2017) Do programmers do change impact analysis in debugging? Empir Softw Eng 22(2):631–669

Jiang J, Wang F, Shen J, Kim S, Kim S (2024a) A survey on large language models for code generation. arXiv preprint arXiv:2406.00515

Jiang M, Liu KZ, Zhong M, Schaeffer R, Ouyang S, Han J, Koyejo S (2024b) Investigating data contamination for pre-training language models. arXiv preprint arXiv:2401.06059

Jiménez ÁB (2024) An evaluation of llm code generation capabilities through graded exercises. arXiv preprint arXiv:2410.16292

Jin H, Huang L, Cai H, Yan J, Li B, Chen H (2024) From llms to llm-based agents for software engineering: A survey of current, challenges and future. arXiv preprint arXiv:2408.02479

Joern (2025) Joern. https://docs.joern.io/

Kargén U, Shahmehri N (2012) Inputtracer: A data-flow analysis tool for manual program comprehension of x86 binaries. In: 2012 IEEE 12th international working conference on source code analysis and manipulation, pp 138–143. https://doi.org/10.1109/SCAM.2012.16

Kargén U, Shahmehri N (2016) Towards accurate binary correspondence using runtime-observed values. In: 2016 IEEE international conference on software maintenance and evolution (ICSME), pp 438–442. https://doi.org/10.1109/ICSME.2016.54

Khan JY, Uddin G (2022) Automatic code documentation generation using gpt-3. In: Proceedings of the 37th IEEE/ACM international conference on automated software engineering, pp 1–6

Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y (2022) Large language models are zero-shot reasoners. Adv Neural Inf Process Syst 35:22199–22213

Konopka M, Talian A, Tvarozek J, Navrat P (2018) Data flow metrics in program comprehension tasks. In: Proceedings of the workshop on eye movements in programming, pp 1–6

Koziolek H, Grüner S, Hark R, Ashiwal V, Linsbauer S, Eskandani N (2024) Llm-based and retrieval-augmented control code generation. In: Proceedings of the 1st international workshop on large language models for code, pp 22–29

LeClair A, Haque S, Wu L, McMillan C (2020) Improved code summarization via a graph neural network. In: 28th ACM/IEEE international conference on program comprehension (ICPC'20)

LeClair A, McMillan C (2019) Recommendations for datasets for source code summarization. In: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, vol 1 (Long and Short Papers), pp 3931–3937

Lee GG, Latif E, Shi L, Zhai X (2023) Gemini pro defeated by gpt-4v: Evidence from education. arXiv preprint arXiv:2401.08660

Lehmann D, Thalakottur M, Tip F, Pradel M (2023) That's a tough call: Studying the challenges of call graph construction for webassembly. In: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, association for computing machinery, ISSTA 2023, pp 892–903, https://doi.org/10.1145/3597926.3598104

Li Y, Choi D, Chung J, Kushman N, Schrittwieser J, Leblond R, Eccles T, Keeling J, Gimeno F, Dal Lago A et al (2022) Competition-level code generation with alphacode. Science 378(6624):1092–1097

Li H, Hao Y, Zhai Y, Qian Z (2023a) The hitchhiker's guide to program analysis: A journey with large language models. arXiv preprint arXiv:2308.00245

Li R, Allal LB, Zi Y, Muennighoff N, Kocetkov D, Mou C, Marone M, Akiki C, Li J, Chim J et al (2023b) Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161

Littman DC, Pinto J, Letovsky S, Soloway E (1987) Mental models and software maintenance. J Syst Softw 7(4):341–355

Liu S, Chen Y, Xie X, Siow JK, Liu Y (2021) Retrieval-augmented generation for code summarization via hybrid gnn. In: International conference on learning representations. https://openreview.net/forum?id=zv-typ1gPxA

Liu Z, Zhang Y, Li P, Liu Y, Yang D (2023a) Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. arXiv preprint arXiv:2310.02170

Liu Z, Zhang Y, Li P, Liu Y, Yang D (2023b) Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization, 2023. arxiv:2310.02170

Liu C, Zhang SD, Ibrahimzada AR, Jabbarvand R (2024a) Codemind: A framework to challenge large language models for code reasoning. arXiv preprint arXiv:2402.09664

Liu F, Liu Y, Shi L, Huang H, Wang R, Yang Z, Zhang L, Li Z, Ma Y (2024b) Exploring and evaluating hallucinations in llm-powered code generation. arXiv preprint arXiv:2404.00971

Liu J, Xia CS, Wang Y, Zhang L (2024c) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Adv Neural Inf Process Syst 36

Liu NF, Lin K, Hewitt J, Paranjape A, Bevilacqua M, Petroni F, Liang P (2024) Lost in the middle: How language models use long contexts. Trans Assoc Comput Linguist 12:157–173

Li Y, Wen H, Wang W, Li X, Yuan Y, Liu G, Liu J, Xu W, Wang X, Sun Y, et al. (2024b) Personal llm agents: Insights and survey about the capability, efficiency and security, 2024. arxiv:2401.05459

Li T, Zhang G, Do QD, Yue X, Chen W (2024a) Long-context llms struggle with long in-context learning. arXiv preprint arXiv:2404.02060

Long Q, Wu Y, Wang W, Pan SJ (2024) Does in-context learning really learn? rethinking how large language models respond and solve tasks via in-context learning. arXiv preprint arXiv:2404.07546

Lu S, Guo D, Ren S, Huang J, Svyatkovskiy A, Blanco A, Clement CB, Drain D, Jiang D, Tang D, Li G, Zhou L, Shou L, Zhou L, Tufano M, Gong M, Zhou M, Duan N, Sundaresan N, Deng SK, Fu S, Liu S (2021) Codexglue: A machine learning benchmark dataset for code understanding and generation. arxiv:2102.04664

Lu G, Ju X, Chen X, Pei W, Cai Z (2024) Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. J Syst Softw 212:112031

Lu S, Bigoulaeva I, Sachdeva R, Madabushi HT, Gurevych I (2024b) Are emergent abilities in large language models just in-context learning? In: Proceedings of the 62nd annual meeting of the association for computational linguistics (vol 1: Long Papers), pp 5098–5139

Luo Y (2025) Can we translate code better with llms and call graph analysis? In: Proceedings of the thirty-fourth international joint conference on artificial intelligence, pp 7625–7633

Luo L, Schäf M, Sanchez D, Bodden E (2021) Ide support for cloud-based static analyses. In: Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, association for computing machinery, ESEC/FSE 2021, pp 1178–1189. https://doi.org/10.1145/3468264.3468535

Ma W, Liu S, Lin Z, Wang W, Hu Q, Liu Y, Zhang C, Nie L, Li L, Liu Y (2023) Lms: Understanding code syntax and semantics for code analysis. arXiv preprint arXiv:2305.12138

Ma B, Li R, Yuanlong W, Tan H, Li X (2025) Memorization$\neq$ understanding: Do large language models have the ability of scenario cognition? In: Proceedings of the 2025 conference on empirical methods in natural language processing, pp 20758–20774

Maalej W, Tiarks R, Roehm T, Koschke R (2014) On the comprehension of program comprehension. ACM Trans Softw Eng Methodol (TOSEM) 23(4):1–37

Madaan A, Zhou S, Alon U, Yang Y, Neubig G (2022) Language models of code are few-shot commonsense learners. arXiv preprint arXiv:2210.07128

Mohsin A, Janicke H, Wood A, Sarker IH, Maglaras L, Janjua N (2024) Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse llms. arXiv preprint arXiv:2406.12513

Mosterman PJ (2006) Automatic code generation: Facilitating new teaching opportunities in engineering education. In: Proceedings. Frontiers in Education. 36th Annual Conference, pp 1–6, https://doi.org/10.1109/FIE.2006.322699

Murphy GC, Notkin D, Griswold WG, Lan ES (1998) An empirical study of static call graph extractors. ACM Trans Softw Eng Methodol 7(2):158–191. https://doi.org/10.1145/279310.279314

Nachtigall M, Nguyen Quang Do L, Bodden E (2019) Explaining static analysis - a perspective. In: 2019 34th IEEE/ACM international conference on automated software engineering workshop (ASEW), pp 29–32. https://doi.org/10.1109/ASEW.2019.00023

Nam D, Macvean A, Hellendoorn V, Vasilescu B, Myers B (2024) Using an llm to help with code understanding. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, association for computing machinery, ICSE '24. https://doi.org/10.1145/3597503.3639187

Neamtiu I, Foster JS, Hicks M (2005) Understanding source code evolution using abstract syntax tree matching. SIGSOFT Softw Eng Notes 30(4):1–5. https://doi.org/10.1145/1082983.1083143

O'brien MP, (2003) Software comprehension-a review & research direction. Department of computer science & information systems university of limerick, Ireland, Technical Report

Pan J, Sadé A, Kim J, Soriano E, Sole G, Flamant S (2023) Stelocoder: a decoder-only llm for multi-language to python code translation. arXiv preprint arXiv:2310.15539

Pan R, Ibrahimzada AR, Krishna R, Sankar D, Wassi LP, Merler M, Sobolev B, Pavuluri R, Sinha S, Jabbarvand R (2024) Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, association for computing machinery, ICSE '24. https://doi.org/10.1145/3597503.3639226

Papineni K, Roukos S, Ward T, Zhu WJ (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th annual meeting on association for computational linguistics, association for computational linguistics, pp 311–318

Park J, Lee H, Ryu S (2021) A survey of parametric static analysis. ACM Comput Surv 54(7). https://doi.org/10.1145/3464457

Peeperkorn M, Kouwenhoven T, Brown D, Jordanous A (2024) Is temperature the creativity parameter of large language models? arXiv preprint arXiv:2405.00492

Pennington N (1987) Stimulus structures and mental representations in expert comprehension of computer programs. Cogn Psychol 19(3):295–341

Poludin M (2025) qlora_for_codegen. https://github.com/poludmik/qlora_for_codegen

Qi Z, Fang Y, Zhang M, Sun Z, Wu T, Liu Z, Lin D, Wang J, Zhao H (2023) Gemini vs gpt-4v: A preliminary comparison and combination of vision-language models through qualitative cases. arXiv preprint arXiv:2312.15011

Rasnayaka S, Wang G, Shariffdeen R, Iyer GN (2024) An empirical study on usage and perceptions of llms in a software engineering project. In: Proceedings of the 1st international workshop on large language models for code, pp 111–118

Rodeghero P, McMillan C (2015) An empirical study on the patterns of eye movement during summarization tasks. In: 2015 ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), pp 1–10

Ross SI, Martinez F, Houde S, Muller M, Weisz JD (2023) The programmer's assistant: Conversational interaction with a large language model for software development. In: Proceedings of the 28th international conference on intelligent user interfaces, association for computing machinery, IUI '23, pp 491–514. https://doi.org/10.1145/3581641.3584037

Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T et al (2023) Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950

Ryder B (1979) Constructing the call graph of a program. IEEE Trans Softw Eng SE-5(3):216–226. https://doi.org/10.1109/TSE.1979.234183

Sarsa S, Denny P, Hellas A, Leinonen J (2022) Automatic generation of programming exercises and code explanations using large language models. In: Proceedings of the 2022 ACM conference on international computing education research, vol 1, pp 27–43

Schanzer E, Bahram S, Krishnamurthi S (2019) Accessible ast-based programming for visually-impaired programmers. In: Proceedings of the 50th ACM technical symposium on computer science education, association for computing machinery, SIGCSE '19, pp 773–779. https://doi.org/10.1145/3287324.3287499

Shneiderman B, Mayer R (1979) Syntactic/semantic interactions in programmer behavior: A model and experimental results. Int J Comput Inf Sci 8:219–238

Siam MK, Gu H, Cheng JQ (2024) Programming with ai: Evaluating chatgpt, Gemini, alphacode, and github copilot for programmers. arXiv preprint arXiv:2411.09224

Siddiqui S, Metta R, Madhukar K (2023) Towards multi-language static code analysis. In: 2023 IEEE 34th International symposium on software reliability engineering workshops (ISSREW), pp 81–82. https://doi.org/10.1109/ISSREW60843.2023.00051

Siegmund J (2016) Program comprehension: Past, present, and future. In: 2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER), vol 5, pp 13–20. https://doi.org/10.1109/SANER.2016.35

Sihler F (2024) Improving the comprehension of r programs by hybrid dataflow analysis. In: Proceedings of the 39th IEEE/ACM international conference on automated software engineering, association for computing machinery, ASE '24, pp 2490–2493, https://doi.org/10.1145/3691620.3695603

Singh D, Sekar VR, Stolee KT, Johnson B (2017) Evaluating how static analysis tools can reduce code review effort. In: 2017 IEEE symposium on visual languages and human-centric computing (VL/HCC), pp 101–105. https://doi.org/10.1109/VLHCC.2017.8103456

Sobo A, Mubarak A, Baimagambetov A, Polatidis N (2025) Evaluating llms for code generation in hri: A comparative study of chatgpt, gemini, and claude. Appl Artif Intell 39(1):2439610

Srcml (2025) srcml tools. https://www.srcml.org/

Stapleton S, Gambhir Y, LeClair A, Eberhart Z, Weimer W, Leach K, Huang Y (2020) A human study of comprehension and code summarization. In: Proceedings of the 28th international conference on program comprehension, pp 2–13

Storey MA (2005) Theories, methods and tools in program comprehension: past, present and future. In: 13th International workshop on program comprehension (IWPC'05), pp 181–191. https://doi.org/10.1109/WPC.2005.38

Strzalkowski P, Strzalkowska A, Chhablani J, Pfau K, Errera MH, Roth M, Schaub F, Bechrakis NE, Hoerauf H, Reiter C et al (2024) Evaluation of the accuracy and readability of chatgpt-4 and google gemini in providing information on retinal detachment: a multicenter expert comparative study. Int J Retina Vitreous 10(1):61

Su CY, McMillan C (2024) Distilled gpt for source code summarization. Autom Softw Eng 31(1):22

Su CY, Bansal A, Jain V, Ghanavati S, Mcmillan C (2023) A language model of java methods with train/test deduplication. arXiv preprint arXiv:2305.08286

Su CY, Bansal A, Huang Y, Li TJJ, McMillan C (2024) Context-aware code summary generation. arXiv preprint arXiv:2408.09006

Su CY, Bansal A, Huang Y, Li TJJ, McMillan C (2025) Context-aware code summary generation. J Syst Softw 112580

Sui L, Dietrich J, Tahir A, Fourtounis G (2020) On the recall of static call graph construction in practice. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, association for computing machinery, ICSE '20, pp 1049–1060. https://doi.org/10.1145/3377811.3380441

Sun Z, Zhu Q, Xiong Y, Sun Y, Mou L, Zhang L (2020) Treegen: A tree-based transformer architecture for code generation. Proc AAAI Conf Artif Intell 34:8984–8991

Sun W, Fang C, Miao Y, You Y, Yuan M, Chen Y, Zhang Q, Guo A, Chen X, Liu Y et al (2023a) Abstract syntax tree for programming language understanding and representation: How far are we? arXiv preprint arXiv:2312.00413

Sun W, Fang C, You Y, Miao Y, Liu Y, Li Y, Deng G, Huang S, Chen Y, Zhang Q et al (2023b) Automatic code summarization via chatgpt: How far are we? arXiv preprint arXiv:2305.12865

Sun Q, Chen Z, Xu F, Cheng K, Ma C, Yin Z, Wang J, Han C, Zhu R, Yuan S et al (2024a) A survey of neural code intelligence: Paradigms, advances and beyond. arXiv preprint arXiv:2403.14734

Sun W, Miao Y, Li Y, Zhang H, Fang C, Liu Y, Deng G, Liu Y, Chen Z (2024b) Source code summarization in the era of large language models. CoRR

Talebirad Y, Nadiri A (2023) Multi-agent collaboration: Harnessing the power of intelligent llm agents. arXiv preprint arXiv:2306.03314

Tang Z, Shen X, Li C, Ge J, Huang L, Zhu Z, Luo B (2022) Ast-trans: Code summarization with efficient tree-structured attention. In: Proceedings of the 44th international conference on software engineering, pp 150–162

Team G, Anil R, Borgeaud S, Alayrac JB, Yu J, Soricut R, Schalkwyk J, Dai AM, Hauth A, Millican K et al (2023) Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805

Team G, Georgiev P, Lei VI, Burnell R, Bai L, Gulati A, Tanzer G, Vincent D, Pan Z, Wang S, et al. (2024) Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530

Tian Y, Ravichander A, Qin L, Le Bras R, Marjieh R, Peng N, Choi Y, Griffiths TL, Brahman F (2024) Thinking out-of-the-box: A comparative investigation of human and llms in creative problem-solving. In: ICML 2024 workshop on LLMs and cognition

Tona C, Juárez-Ramírez R, Jiménez S, Durán M (2024) Exploring llm tools through the eyes of industry experts and novice programmers. In: 2024 12th International conference in software engineering research and innovation (CONISOFT), pp 313–321. https://doi.org/10.1109/CONISOFT63288.2024.00048

Tymchuk Y, Ghafari M, Nierstrasz O (2018) Jit feedback: what experienced developers like about static analysis. In: Proceedings of the 26th conference on program comprehension, association for computing machinery, ICPC '18, pp 64–73. https://doi.org/10.1145/3196321.3196327
Vassallo C, Panichella S, Palomba F, Proksch S, Zaidman A, Gall HC (2018) Context is king: The developer perspective on the usage of static analysis tools. In: 2018 IEEE 25th International conference on software analysis, evolution and reengineering (SANER), pp 38–49. https://doi.org/10.1109/SANER.2018.8330195
Vassallo C, Panichella S, Palomba F, Proksch S, Gall HC, Zaidman A (2020) How developers engage with static analysis tools in different contexts. Empir Softw Eng 25:1419–1457
Wallace R, Bansal A, Karas Z, Tang N, Huang Y, Li TJJ, McMillan C (2025) Programmer visual attention during context-aware code summarization. IEEE Trans Softw Eng
Walunj V, Gharibi G, Ho DH, Lee Y (2019) Graphevo: Characterizing and understanding software evolution using call graphs. In: 2019 IEEE international conference on big data (Big Data), pp 4799–4807. https://doi.org/10.1109/BigData47090.2019.9005560
Wang Y, Wang W, Joty S, Hoi SC (2021) Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv preprint arXiv:2109.00859
Wang X, Li S, Ji H (2022) Code4struct: Code generation for few-shot event structure prediction. arXiv preprint arXiv:2210.12810
Wang B, Yue X, Sun H (2023) Can chatgpt defend its belief in truth? Evaluating llm reasoning via debate. arXiv preprint arXiv:2305.13160
Wang C, Zhang W, Su Z, Xu X, Xie X, Zhang X (2024) Llmdfa: analyzing dataflow in code with large language models. Adv Neural Inf Process Syst 37:131545–131574
Wang J, Luo X, Cao L, He H, Huang H, Xie J, Jatowt A, Cai Y (2024b) Is your ai-generated code really safe? Evaluating large language models on secure code generation with codeseceval. arXiv preprint arXiv:2407.02395
Wang T, Zhou N, Chen Z (2024c) Enhancing computer programming education with llms: A study on effective prompt engineering for python code generation. arXiv preprint arXiv:2407.05437
Weideman N, Arasteh S, Raghothaman M, Mirkovic J, Hauser C (2025) Data flows in you: Benchmarking and improving static data-flow analysis on binary executables. arXiv preprint arXiv:2506.00313
Welty C (1997) Augmenting abstract syntax trees for program understanding. In: Proceedings 12th IEEE international conference automated software engineering, pp 126–133. https://doi.org/10.1109/ASE.1997.632832
Weyssow M, Zhou X, Kim K, Lo D, Sahraoui H (2025) Exploring parameter-efficient fine-tuning techniques for code generation with large language models. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3714461
Wohlin C, Runeson P, Höst M, Ohlsson MC, Regnell B, Wesslén A (2012) Experimentation in software engineering. Springer Science & Business Media
Wu X, Duan R, Ni J (2024) Unveiling security, privacy, and ethical concerns of chatgpt. J Inf Intell 2(2):102–115
Xie D, Zheng M, Liu X, Wang J, Wang C, Tan L, Zhang X (2025) Core: Benchmarking llms code reasoning capabilities through static analysis tasks. arXiv preprint arXiv:2507.05269
Xu FF, Vasilescu B, Neubig G (2022) In-ide code generation from natural language: Promise and challenges. ACM Trans Softw Eng Methodol 31(2). https://doi.org/10.1145/3487569
Xue Z, Li L, Tian S, Chen X, Li P, Chen L, Jiang T, Zhang M (2024) Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing. In: Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis, association for computing machinery, ISSTA 2024, pp 1643–1655. https://doi.org/10.1145/3650212.3680388
Yan W, Tian Y, Li Y, Chen Q, Wang W (2023) Codetransocean: A comprehensive multilingual benchmark for code translation. arXiv preprint arXiv:2310.04951
Yang K, Liu J, Wu J, Yang C, Fung YR, Li S, Huang Z, Cao X, Wang X, Wang Y, et al. (2024) If llm is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. arXiv preprint arXiv:2401.00812
Yin X, Ni C, Wang S, Li Z, Zeng L, Yang X (2024) Thinkrepair: Self-directed automated program repair. In: Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis, association for computing machinery, ISSTA 2024, pp 1274–1286. https://doi.org/10.1145/3650212.3680359
Yu (2023) c-to-rust. https://huggingface.co/datasets/yijunyu/c-to-rust
Yuan Z, Liu J, Zi Q, Liu M, Peng X, Lou Y (2023) Evaluating instruction-tuned large language models on code comprehension and generation. arXiv preprint arXiv:2308.01240
Yuan Z, Chen W, Wang H, Yu K, Peng X, Lou Y (2024) Transagent: An llm-based multi-agent system for code translation. arXiv preprint arXiv:2409.19894

Zamfirescu-Pereira JD, Wong RY, Hartmann B, Yang Q (2023) Why johnny can't prompt: how non-ai experts try (and fail) to design llm prompts. In: Proceedings of the 2023 CHI conference on human factors in computing systems, pp 1–21

Zhang J, Wang X, Zhang H, Sun H, Wang K, Liu X (2019) A novel neural source code representation based on abstract syntax tree. In: Proceedings of the 41st international conference on software engineering, IEEE Press, pp 783–794

Zhang C, Wang J, Zhou Q, Xu T, Tang K, Gui H, Liu F (2022) A survey of automatic source code summarization. Symmetry 14(3):471

Zhang Z, Chen C, Liu B, Liao C, Gong Z, Yu H, Li J, Wang R (2023) A survey on language models for code. arXiv preprint arXiv:2311.07989

Zhang X, Hou X, Qiao X, Song W (2024) A review of automatic source code summarization. Empir Softw Eng 29(6):162

Zhang R, Zhang S, Xie L (2026) A systematic exploration of c-to-rust code translation based on large language models: prompt strategies and automated repair. Autom Softw Eng 33(1):21

Zhao Y, Zhang R, Li W, Huang D, Guo J, Peng S, Hao Y, Wen Y, Hu X, Du Z et al (2024) Assessing and understanding creativity in large language models. arXiv preprint arXiv:2401.12491

Zheng Z, Ning K, Wang Y, Zhang J, Zheng D, Ye M, Chen J (2023) A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv preprint arXiv:2311.10372

Zheng Z, Ning K, Zhong Q, Chen J, Chen W, Guo L, Wang W, Wang Y (2025) Towards an understanding of large language models in software engineering tasks. Empir Softw Eng 30(2):50

Zhong L, Wang Z (2024) Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. Proc AAAI Conf Artif Intell 38:21841–21849

Zhou S, Alon U, Agarwal S, Neubig G (2023) Codebertscore: Evaluating code generation with pretrained models of code. arXiv preprint arXiv:2302.05527

Zhu M, Jain A, Suresh K, Ravindran R, Tipirneni S, Reddy CK (2022) Xlcost: A benchmark dataset for cross-lingual code intelligence. arXiv preprint arXiv:2206.08474